Test Automation Framework Guide

A complete guide to one hybrid framework for web, mobile, desktop and API testing, with system design principles, design patterns, risk analysis and anti-flakiness. Code in C#, TypeScript, Python and Java.

Author
Hasin Md. Daiyan

Contact
daiyanhasin07@gmail.com

This is the print edition. The HTML edition has the interactive diagrams, the flaky-test simulator and the run-time calculator.

Contents

Start here

1 What a test automation framework is

2 Build order: start small, grow on purpose

Plan

3 The test pyramid: choose the right level

4 Risk analysis: decide what to automate first

Architecture

5 What "hybrid framework" means

6 Layered architecture and the dependency rule

7 Folder structure and naming

8 System design principles for test code

9 SOLID in a test framework

Building blocks

10 Configuration, environments and secrets

11 Driver interface and driver factory

12 Base page and the Template Method pattern

13 Page Object Model (POM) across platforms

14 Facade: business flows in one call

15 Strategy: swap behavior at runtime

16 Test data: JSON, Excel, CSV, database, API

17 Hooks, fixtures and the test lifecycle

18 BDD: feature files and step definitions

19 API layer: clients, checks and fast setup

20 Utilities: small, focused helpers

Reliability

21 Locator strategy for web, mobile and desktop

22 Anti-flakiness: why tests pass, then fail, with no code change

23 Waiting and synchronization, in code

24 Retries, flaky tracking and quarantine

Delivery

25 Reporting, observability and the Observer pattern

26 Parallel execution and CI/CD

27 Security and production safety

28 Review checklist and definition of done

1 What a test automation framework is

A framework is the shared structure around your tests. It lets many people write tests the same way, on web, mobile, desktop and API.

What it is

There are three levels. Many people mix them up.

Test script Automates one check. For example: "a customer can log in."

Test suite A group of related scripts. For example: all login and checkout tests.

Framework The structure every script uses: configuration, drivers, page objects, test data, waits, hooks, reports and CI.

Why you need it

Without a framework, each test repeats the same setup. URLs, locators, passwords and waits get copied into hundreds of files. Then one small UI change breaks hundreds of tests, and nobody trusts the results.

With a framework, each piece of knowledge lives in one place. A locator lives in one page object. A URL lives in one config file. A wait lives in one driver adapter. Change it once and every test gets the fix.

How this guide works

Every topic follows the same order, so you always know where you are:

1. **Read** what it is and why it matters.
2. **See** a diagram or a small interactive example.
3. **Code** the same idea in C#, TypeScript, Python and Java. Pick your language once at the top of the page.
4. **Watch out** for the common mistakes.

The reference stack is Playwright for web and API, Appium for Android, iOS and Windows desktop, and each language's normal test runner. Reqnroll, Cucumber and pytest-bdd are optional for BDD.

Code: the same test, before and after

C#

```
// BAD: URL, locators, data, secrets and waits all live in the test
[Test]
public async Task Login()
{
    await Page.GotoAsync("https://staging.shop.example/login");
    await Page.FillAsync("#email", "admin@shop.example");
    await Page.FillAsync("#pwd", "Admin123");   // secret in code
    await Page.ClickAsync("//div[3]/button");   // fragile XPath
    await Task.Delay(5000);                     // hard sleep
}

// GOOD: the test only says what matters
```

```
[Test, Category("smoke")]
public async Task Admin_can_open_the_dashboard()
{
    await LoginFacade.LoginAsAsync(Users.Get("admin"));
    await Dashboard.ExpectLoadedAsync();
}
```

TypeScript

```
// BAD: URL, locators, data, secrets and waits all live in the test
test('login', async ({ page }) => {
  await page.goto('https://staging.shop.example/login');
  await page.fill('#email', 'admin@shop.example');
  await page.fill('#pwd', 'Admin123');      // secret in code
  await page.click('//div[3]/button');      // fragile XPath
  await page.waitForTimeout(5000);          // hard sleep
});

// GOOD: the test only says what matters
test('admin can open the dashboard @smoke', async ({ loginFacade, users, dashboard }) => {
  await loginFacade.loginAs(users.get('admin'));
  await dashboard.expectLoaded();
});
```

Python

```
# BAD: URL, locators, data, secrets and waits all live in the test
def test_login(page):
    page.goto("https://staging.shop.example/login")
    page.fill("#email", "admin@shop.example")
    page.fill("#pwd", "Admin123")            # secret in code
    page.click("//div[3]/button")            # fragile XPath
    page.wait_for_timeout(5000)              # hard sleep


# GOOD: the test only says what matters
@pytest.mark.smoke
def test_admin_can_open_the_dashboard(login_facade, users, dashboard):
    login_facade.login_as(users.get("admin"))
    dashboard.expect_loaded()
```

Java

```
// BAD: URL, locators, data, secrets and waits all live in the test
@Test
void login() throws InterruptedException {
    page.navigate("https://staging.shop.example/login");
    page.fill("#email", "admin@shop.example");
    page.fill("#pwd", "Admin123");          // secret in code
    page.click("//div[3]/button");          // fragile XPath
    Thread.sleep(5000);                     // hard sleep
}

// GOOD: the test only says what matters
@Test
@Tag("smoke")
void adminCanOpenTheDashboard() {
    loginFacade.loginAs(users.get("admin"));
    dashboard.expectLoaded();
}
```

Watch out

> **!** Automation does not replace exploratory testing. It repeats known checks fast. It does not find new kinds of bugs by itself.

! A framework is not "more layers is better." Start small and add a layer only when it fixes a problem you can name.

2 Build order: start small, grow on purpose

You do not need every pattern on day one. Build in this order and stop at each step until it works.

Why the order matters

Teams often build ten base classes and a custom report before they have one working test. That framework is hard to learn and hard to change. A working smoke test in CI on day two teaches you more than any design document.

See it

1. **List platforms and risks**
   Web? Android? iOS? Windows desktop? API? Which flows would hurt most if they broke?
2. **Choose one language and one runner**
   Pick what your team already maintains with confidence.
3. **Write one smoke test that runs locally**
   Open the app and check one thing. Nothing else.
4. **Add validated configuration**
   Base URL and platform from the environment. Fail fast when something is missing.
5. **Add page objects when duplication appears**
   The second time you copy a locator, move it into a page.
6. **Create and clean up test data**
   Unique data per test, created through the API.
7. **Capture failure evidence**
   Screenshot, trace and log on failure, in one output folder.
8. **Run the same command in CI**
   If CI runs a different command than you do, results will differ.
9. **Add parallel runs after tests are isolated**
   Parallel runs expose shared state. Fix isolation first.
10. **Add patterns only for real problems**
    Facade, strategy, observer: add each one when you can describe the pain it removes.

Milestone: a new teammate can clone, install, configure, run the smoke test and read a failure, without asking anyone for hidden rules.

3 The test pyramid: choose the right level

Test each behavior at the lowest level that gives you enough confidence.

What it is

The pyramid is a picture of a healthy test mix. The bottom has many fast, cheap tests. The top has a few slow, realistic ones.

Why you need it

UI tests are slow and break more easily, because they depend on the browser, the network, the layout and the data. If most of your tests are UI tests, the suite becomes slow and flaky. This is often called an "ice-cream cone," the pyramid upside down.

See it: click a level

End-to-end / UI

A real user journey through a browser, phone or desktop app.

How many	Few
Speed	Slow: seconds to minutes each
Breaks	Most often: UI, timing, data, devices
Examples	Customer completes checkout. Login works on Android and iOS.
Tools	Playwright, Appium, Cucumber/Reqnroll on top

API / integration / contract

Calls the backend directly. Checks business rules, permissions and contracts between services.

How many	More
Speed	Fast: milliseconds to seconds
Breaks	Sometimes: service and data issues
Examples	Discount rules, refunds, unauthorized access is rejected, schema checks.
Tools	Playwright request, REST clients, Pact for contracts

Unit / component / static

Checks one function or component in isolation. Usually written by developers.

How many	Many
Speed	Fastest: milliseconds
Breaks	Rarely: only when the code changes
Examples	Price calculation, validators, date formatting, a single UI component.
Tools	xUnit/NUnit, Jest/Vitest, pytest, JUnit, linters and type checks

Exploratory and manual testing (above the pyramid)

People find new kinds of problems no script was written for: confusing flows, odd edge cases, things that feel wrong.

How to choose the level

For each behavior, ask these questions in order. Stop at the first "yes."

1. Can a unit test check it completely? Use a unit test.
2. Can an API, component or contract test check it? Use that.
3. Does it need a real browser, device or full user journey? Use a UI or end-to-end test.
4. Does automating it cost more than the risk it removes? Keep it manual or exploratory.

Behavior	Best level
Discount calculation	Unit or API
Unauthorized users are rejected	API
Service A still accepts Service B's messages	Contract
Login form works with the keyboard	UI plus accessibility check
Customer completes checkout	A few end-to-end tests
Button spacing and alignment	Visual check or manual review

Watch out

! The pyramid is a guide, not a quota. A microservice platform may need more contract tests. A content site may need more visual and accessibility checks.

! Do not test business rules through the UI when the API can check them in milliseconds.

4 Risk analysis: decide what to automate first

You can never automate everything. Risk analysis tells you where your effort protects the most.

What it is

Risk is two things multiplied together:

Risk = Likelihood × Impact

Likelihood is how likely the feature is to break. Complex code, frequent changes, new developers and third-party dependencies raise it. **Impact** is how bad it is when it breaks. Lost money, blocked users, legal problems and data loss raise it.

Why you need it

Without risk analysis, teams automate what is easy, not what matters. They end up with 200 tests for the profile page and none for refunds. Risk analysis also gives you a clear answer when a manager asks, "Why did you test this and not that?"

How to do it

1. List features and flows. Use the user stories, the menu, or the API list.
2. Score each one from 1 to 5 for likelihood and for impact. Do it with a developer and a product person, not alone.
3. Multiply the two scores. Sort from highest to lowest.
4. Decide the depth for each band: high risk gets several levels of tests on every pull request, low risk may stay manual.
5. Review the scores every release. Risk changes when code changes.

See it: score a feature

Click a feature on the grid, or set your own scores. The panel shows what to do with it.

Feature	Likelihood	Impact	Score	Band
Checkout flow	4	5	**20**	Critical
Promo codes	4	4	**16**	Critical
Payments & refunds	3	5	**15**	Critical
Login & sessions	3	5	**15**	Critical
Search & filters	4	3	**12**	High
Push notifications	4	2	**8**	Medium
Order history	2	3	**6**	Medium
Admin CSV export	2	2	**4**	Low

Feature	Likelihood	Impact	Score	Band
Profile photo upload	3	1	**3**	Low
Footer links	1	1	**1**	Low

Band	Score	What to do
Critical	15 to 25	Automate first. Cover every business rule with unit and API tests, and add one or two end-to-end journeys per platform. Run them on every pull request as part of smoke.
High	10 to 14	Automate soon. API tests for the rules plus one UI path. Run on every merge to main.
Medium	5 to 9	Automate the main path once the feature is stable. Run in the nightly regression.
Low	1 to 4	Keep it manual or exploratory. Automate only if it is cheap and stable.

See it: should this be automated?

Will this check run many times? (every build, every release)

> No → test it manually or exploratory

yes

Is the feature stable enough that the test will not change every week?

> No → wait, write it after the design settles

yes

Can the result be checked by a machine? (not "does it look nice")

> No → visual review or manual

yes

Is the risk score medium or high?

> No → low priority; automate later if time allows

yes

Automate it, at the lowest level that gives confidence.

Watch out

! Do not score alone. QA sees likelihood well. Product sees impact well. You need both.

! A high score does not always mean "more UI tests." It often means "more API and unit tests, plus one end-to-end journey."

5 What "hybrid framework" means

A hybrid framework mixes several styles of automation and several platforms in one shared structure.

What it is

"Hybrid" has two meanings, and a modern framework usually has both:

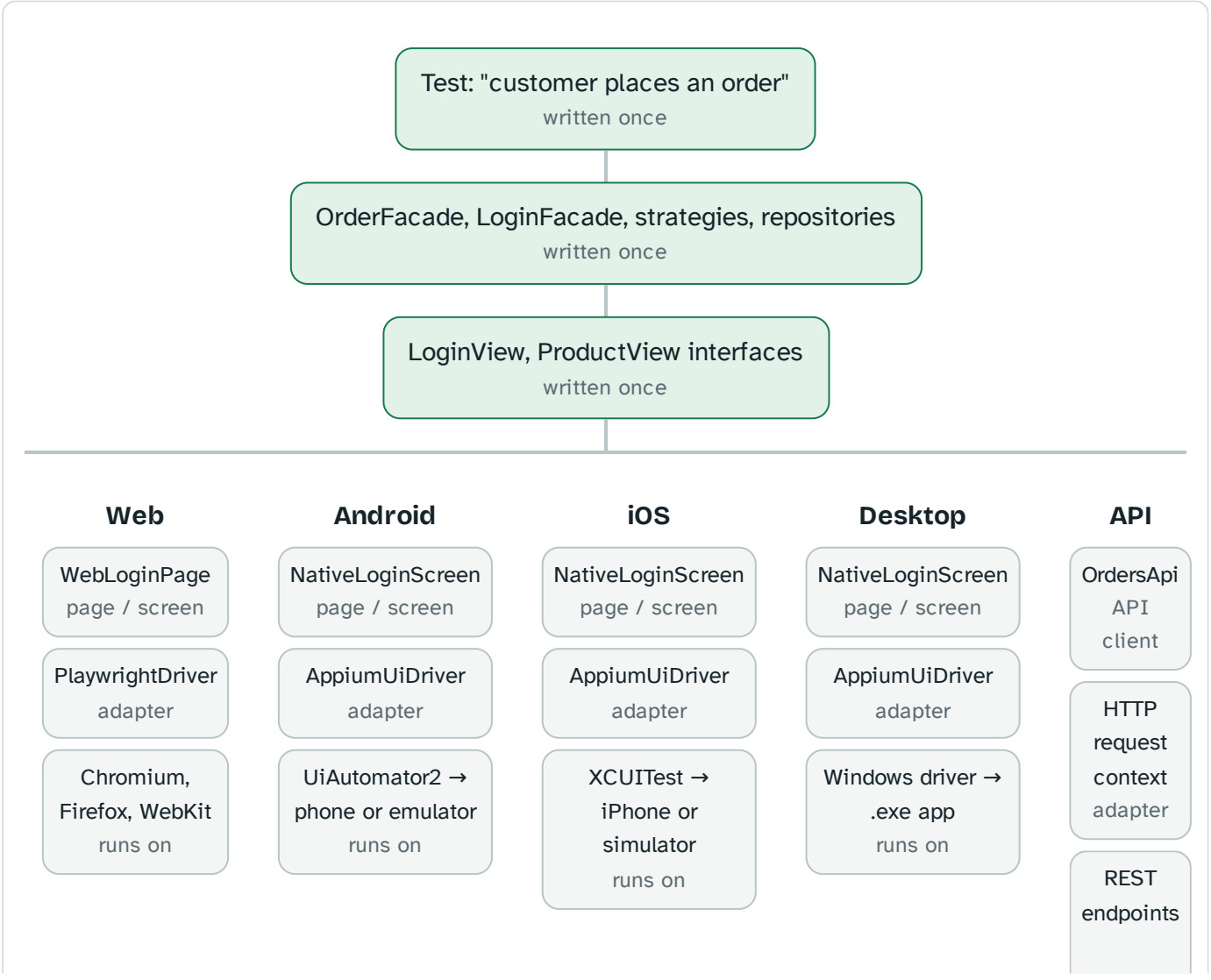
Hybrid in style — It combines a **modular** design (page objects, facades), **data-driven** tests (one test, many rows from JSON or Excel), and optional **BDD** (Gherkin feature files on top).

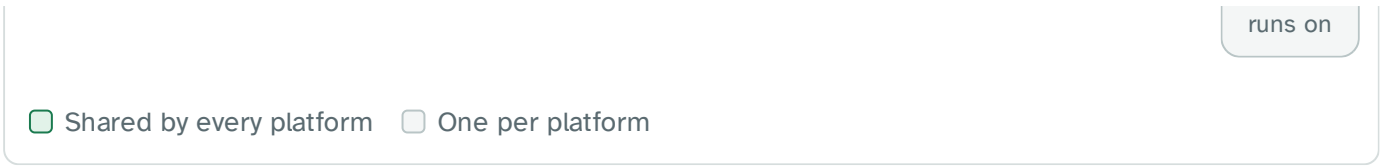
Hybrid in platform — One framework drives **web** (Playwright), **mobile** (Appium for Android and iOS), **desktop** (Appium Windows driver, or FlaUI and pywinauto) and **API** tests.

Why you need it

Many products have a web app, a mobile app and an admin desktop tool that all share the same business flows. If you build three separate frameworks, you write login, test data, config and reporting three times. In a hybrid framework, the business layer is written once and only the bottom layer changes per platform.

See it: one test, many platforms





How to do it

1. Write tests against **business actions** (facades), not against screens.
2. Give each platform its own **page objects**, because screens differ.
3. Hide each automation tool behind one small **driver interface**. Playwright and Appium become two adapters.
4. Pick the platform at runtime from **configuration** (`PLATFORM=android`), using a **factory**.

Watch out

> **!** Do not force one page object to serve every platform with `if (android) ... else ...` inside every method. Use one page class per platform, behind one interface.

> **!** For desktop: Microsoft's WinAppDriver, which the Appium Windows driver uses, is no longer actively developed. Check its status before you commit to it. FlaUI (C#) and pywinauto (Python) are common alternatives. For Electron apps, Playwright can drive the app directly.

6 Layered architecture and the dependency rule

Each layer has one job. Calls only go downward.

What it is

The framework is a stack of layers. A layer may call the layers below it, never the layers above. Some parts, like configuration and logging, are used by every layer. These are called **cross-cutting** concerns.

Why you need it

When calls go in every direction, a change anywhere can break anything. With one direction, you know exactly what a change can affect. A locator change touches only the page layer. A switch from Appium to another tool touches only the driver layer.

See it: click a layer

Layer (top to bottom)	Job	May call	Must not
Tests `tests/`	Describe behavior and check outcomes.	Facades, pages, API clients, repositories	Locators, URLs, sleeps, creating drivers
Step definitions (BDD) `tests/steps/`	Translate Gherkin sentences into framework calls.	Facades, repositories	Business logic, locators, if/else decisions
Facades `framework/facades/`	Run one business flow across several pages or APIs.	Pages, strategies, API clients	Locators, the final business assertion
Strategies `framework/strategies/`	Interchangeable ways to do one job, such as UI login or API login.	Pages, API clients	Choosing themselves (the facade or config chooses)
Pages and API clients `framework/pages/`, `framework/api/`	Own locators and endpoints. Offer actions like loginAs().	The driver interface or the HTTP client	Reading test data, business assertions, creating drivers
Base page `framework/pages/base/`	Fix the open-then-wait steps for every page (Template Method).	The driver interface	Page-specific locators, a long list of random helpers
Driver interface and adapters `framework/drivers/`	Hide Playwright and Appium behind one small interface. Own all waiting.	The real tool: Playwright, Appium, WebdriverIO	Knowing about pages, tests or business flows
Driver factory `framework/factory/`	Read the config and create the right adapter.	Adapters, config	Running test steps
Tools and devices `(outside the repo)`	Browsers, emulators, simulators, real devices, desktop apps, APIs.	-	-
Cross-cutting: used by every layer			
Configuration `framework/config/`	One read-only source for environment, platform, URLs and timeouts.	Environment variables, env files	Holding secrets in Git, being changed by tests

Layer (top to bottom)	Job	May call	Must not
Test data `framework/data/, testdata/`	Repositories, builders and readers for JSON, Excel, CSV, database and API data.	Files, database, API	Being read directly by tests or pages
Logging and utils `framework/utils/`	Small shared helpers: unique data, polling, masking, logging.	Nothing above them	Hidden state, locators, business flows
Reporting and listeners `framework/listeners/, output/`	Collect evidence and publish results (Observer).	Driver for screenshots, file system	Failing the run when a listener breaks

Rules to follow

- **Tests** describe behavior and check outcomes. No locators, no URLs, no sleeps.
- **Step definitions** (BDD only) translate Gherkin into calls. No logic.
- **Facades** run business flows across several pages or APIs.
- **Pages** own locators and screen actions.
- **Drivers** own the automation tool and the waiting.
- Pages may expose technical checks like `expectLoaded()`. Business checks stay visible in the test, so a reader can see what is being tested.
- A facade may skip layers when that is its documented job. For example, it may use an API client for setup. Do not force every call through every layer.

7 Folder structure and naming

The folder tree is the architecture you can see. A new person should guess where a file lives without asking.

What it is

The structure keeps three things apart: **reusable framework code**, **tests**, and **data and output**. Inside the framework, each folder matches one layer or one pattern. Web, mobile and desktop each get their own subfolder where they really differ: drivers and pages.

Why you need it

When framework code and tests are mixed, people copy helpers into test files and the framework slowly rots. A clear tree also makes code review easy: "Why is there a locator in `tests/`?"

See it: click a folder

Folder	What goes here	Never put here
`automation-framework/`	The repository root. Keep it boring and predictable.	Loose scripts and copies of other projects.
`README.md`	How to install, configure and run. The first file a new teammate reads.	Out-of-date commands.
`framework/`	Reusable code that tests use. No tests live here.	Test methods, scenario data, generated files.
`config/`	The config manager (Singleton). Loads and validates settings.	Secret values.
`drivers/`	The driver interface and one adapter per tool. All waiting lives here.	Page locators, business logic.
`web/`	Playwright adapter.	Appium code.
`mobile/`	Appium adapter for Android and iOS, with explicit waits.	Sleeps.
`desktop/`	Windows or macOS desktop adapter (Appium Windows driver, FlaUI, pywinauto).	Web-only methods like goTo().
`factory/`	Driver factory and page factory. Creates the right object for the platform.	Test steps.
`pages/`	Page objects, one folder per platform where screens differ.	Assertions about business outcomes, test data.
`base/`	BasePage with the template method open().	Dozens of unrelated helpers.
`web/`	Web page objects using data-testid and role locators.	Native locators.
`mobile/`	Android and iOS screens using accessibility ids.	CSS selectors.
`desktop/`	Desktop windows using AutomationId.	URLs.
`components/`	Pieces shared by many screens: header, menu, date picker.	Full screens.
`facades/`	Business flows across pages and APIs.	Locators.

Folder	What goes here	Never put here
`strategies/`	Interchangeable behaviors behind one interface.	One class per platform-and-method combination.
`api/`	API clients, request and response models, JSON schemas.	UI code.
`clients/`	One client per service or endpoint group.	Assertions on business meaning.
`models/`	Typed request and response shapes.	Logic.
`schemas/`	JSON schemas for contract checks.	Example data.
`data/`	Repositories, builders and file readers (JSON, Excel, CSV).	The data files themselves (those live in testdata/).
`listeners/`	Observers that react to test events: evidence, notifications.	Code that can fail the run.
`utils/`	Small, stateless helpers.	Hidden state, locators, "misc" code.
`tests/`	The test scenarios. Short, readable, one behavior each.	Locators, driver creation, sleeps.
`web/`	Web UI tests.	Mobile-only checks.
`mobile/`	Android and iOS tests, often the same tests run with PLATFORM=android or ios.	Web URLs.
`desktop/`	Desktop app tests.	Browser code.
`api/`	API tests: rules, auth, errors, schemas.	Browser setup.
`features/` (BDD only)	Gherkin .feature files.	Selectors, technical steps.
`steps/` (BDD only)	Step definitions: thin translation from words to calls.	Logic, locators, static state.
`hooks/`	Setup and teardown: driver per test, evidence on failure. Called fixtures or conftest.py in some runners.	Business assertions.
`testdata/`	Non-secret data files that tests read through repositories.	Passwords, tokens, real customer data.
`json/`	Reference data and data-driven cases.	Secrets.
`excel/`	Spreadsheets that business users maintain.	The only copy of important data (hard to review).
`csv/`	Simple flat tables.	Nested data.
`config/`	Non-secret environment files.	Secrets. Those live in .env (git-ignored) or the CI secret store.
`ci/`	Pipeline definitions and scripts.	Hard-coded credentials.
`docs/adr/`	Architecture Decision Records: short notes on why you chose something.	Long design essays nobody updates.
`output/` (git-ignored)	Everything the run generates: reports, screenshots, traces, videos, logs, JUnit XML.	Anything you want to keep in Git.

Code: the real layout in your language

C#

```
Shop.Automation/
├── Shop.Automation.sln
├── src/Shop.Automation.Framework/        # reusable code
│   ├── Config/        TestConfig.cs
│   ├── Drivers/       IUiDriver.cs, Web/PlaywrightDriver.cs,
│   │                  Native/AppiumUiDriver.cs
│   ├── Factory/       DriverFactory.cs, Pages.cs
│   ├── Pages/         Base/BasePage.cs, Web/, Mobile/, Desktop/
│   ├── Facades/       LoginFacade.cs, OrderFacade.cs
│   ├── Strategies/    ILoginStrategy.cs, UiLoginStrategy.cs
│   ├── Api/           Clients/, Models/, Schemas/
│   ├── Data/          UserRepository.cs, JsonUserSource.cs,
│   │                  ExcelReader.cs, Builders/OrderBuilder.cs
│   ├── Listeners/     TestEvents.cs, ScreenshotOnFailure.cs
│   └── Utils/         Poll.cs, Unique.cs, Secrets.cs
├── tests/Shop.Automation.Tests/
│   ├── Web/  Mobile/  Desktop/  Api/
│   ├── Features/          # Reqnroll .feature files (BDD only)
│   ├── StepDefinitions/   # Reqnroll bindings (BDD only)
│   └── Hooks/             BaseTest.cs, BrowserHooks.cs
├── testdata/          json/  excel/  csv/
├── config/            qa.env, staging.env
├── docs/adr/
└── output/            # generated, git-ignored
```

TypeScript

```
shop-automation/
├── package.json, package-lock.json, .nvmrc
├── playwright.config.ts
├── framework/                  # reusable code
│   ├── config/        config.ts
│   ├── drivers/       ui-driver.ts, web/playwright-driver.ts,
│   │                  native/appium-driver.ts
│   ├── factory/       driver-factory.ts, pages.ts
│   ├── pages/         base/base-page.ts, web/, mobile/, desktop/
│   ├── facades/       login-facade.ts, order-facade.ts
│   ├── strategies/    login-strategy.ts
│   ├── api/           clients/, models/, schemas/
│   ├── data/          user-repository.ts, excel-reader.ts,
│   │                  builders/order-builder.ts
│   ├── listeners/     test-events.ts
│   └── utils/         unique.ts, mask.ts
├── tests/
│   ├── web/  mobile/  desktop/  api/
│   ├── features/      # Cucumber .feature files (BDD only)
│   ├── steps/         # step definitions + world.ts (BDD only)
│   └── fixtures.ts    # Playwright fixtures = hooks
├── testdata/          json/  excel/  csv/
├── config/            qa.env, staging.env
├── docs/adr/
└── output/            # generated, git-ignored
```

Python

```
shop-automation/
├── pyproject.toml, requirements.txt
├── framework/                  # reusable code (an importable package)
│   ├── config/        settings.py
│   ├── drivers/       ui_driver.py, web/playwright_driver.py,
│   │                  native/appium_driver.py
│   ├── factory/       driver_factory.py, pages.py
│   ├── pages/         base/base_page.py, web/, mobile/, desktop/
│   ├── facades/       login_facade.py, order_facade.py
│   ├── strategies/    login_strategy.py
│   ├── api/           clients/, models/, schemas/
│   ├── data/          user_repository.py, excel_reader.py,
│   │                  builders/order_builder.py
│   ├── listeners/     test_events.py
```

```
│   └── utils/          unique.py, poll.py, mask.py
├── tests/
│   ├── web/  mobile/  desktop/  api/
│   ├── features/      # pytest-bdd .feature files (BDD only)
│   ├── steps/         # step functions (BDD only)
│   └── conftest.py    # fixtures = hooks
├── testdata/          json/  excel/  csv/
├── config/            qa.env, staging.env
├── docs/adr/
└── output/            # generated, git-ignored
```

Java

```
shop-automation/
├── pom.xml
├── src/main/java/com/shop/framework/     # reusable code
│   ├── config/        Config.java, Platform.java
│   ├── drivers/       UiDriver.java, web/PlaywrightDriver.java,
│   │                  nativeapp/AppiumUiDriver.java
│   ├── factory/       DriverFactory.java, Pages.java
│   ├── pages/         base/BasePage.java, web/, mobile/, desktop/
│   ├── facades/       LoginFacade.java, OrderFacade.java
│   ├── strategies/    LoginStrategy.java, UiLoginStrategy.java
│   ├── api/           clients/, models/, schemas/
│   ├── data/          UserRepository.java, ExcelReader.java,
│   │                  builders/OrderBuilder.java
│   ├── listeners/     TestEvents.java
│   └── utils/         Unique.java, Poll.java, Secrets.java
├── src/test/java/com/shop/tests/
│   ├── web/  mobile/  desktop/  api/
│   ├── steps/         # Cucumber step definitions (BDD only)
│   └── hooks/         BaseTest.java, EvidenceOnFailure.java
├── src/test/resources/
│   ├── features/      # Cucumber .feature files (BDD only)
│   └── junit-platform.properties
├── testdata/          json/  excel/  csv/
├── config/            qa.env, staging.env
├── docs/adr/
└── output/            # generated, git-ignored
```

Naming rules

Item	Rule	Example
Page class	Noun + Page (web) or Screen (native)	WebLoginPage, NativeLoginScreen
Facade	Business noun + Facade	OrderFacade
Strategy	Behavior + Strategy	ApiSessionStrategy
Repository	Data noun + Repository	UserRepository
API client	Service + Api	OrdersApi
Builder	Noun + Builder	OrderBuilder
Test name	Describes the behavior	registered customer can log in
Test ID	Lowercase, hyphens, purpose	checkout-submit
Feature file	Business area, hyphens	customer-login.feature

Use your language's normal file naming: `LoginPage.cs`, `login-page.ts`, `login_page.py`, `LoginPage.java`. Write the rule in the README.

Watch out

- One output folder only. Reports, screenshots, traces, videos and logs all go to `output/`, and Git ignores it.
- Also git-ignore `.env`, login state files, `bin/`, `obj/`, `node_modules/`, `.venv/` and `target/`.
- Do not create empty folders "for later." Add a folder when the first file needs it.

8 System design principles for test code

The same rules that keep large software systems healthy also keep a test framework healthy.

What they are

Principles are rules of thumb, not laws. Each one answers a simple question about your code. Some are about how you write classes, like SRP and DRY. Others are qualities the whole framework should have, like scalability and observability.

Why you need them

Test code grows fast. A suite of 100 tests becomes 2,000 in a year. Without principles, every new test makes the next one harder to write. With them, the cost of a new test stays small.

See it: click a principle

How you write code

Separation of concerns

Different jobs live in different parts of the framework.

In this framework: Tests, pages, data, drivers and reports each have their own layer and folder.

Warning sign: A test file that creates a driver, reads Excel and writes a report.

Healthy version: Each layer does one kind of job and calls the layer below.

Single responsibility (SRP)

A class has one reason to change.

In this framework: LoginPage changes only when the login screen changes.

Warning sign: A LoginPage that also loads users and writes reports.

Healthy version: LoginPage, UserRepository and ReportWriter are separate classes.

Loose coupling

Parts depend on small interfaces, not on each other's details.

In this framework: Pages use the UiDriver interface, so Playwright and Appium can be swapped.

Warning sign: Pages that call Appium directly, so web pages cannot reuse them.

Healthy version: Pages receive a UiDriver through the constructor.

High cohesion

Things that change together live together.

In this framework: Everything about the login screen is in LoginPage.

Warning sign: Login locators spread across five helper files.

Healthy version: One page object per screen.

DRY (don't repeat yourself)

Each piece of knowledge exists once.

In this framework: A locator lives in one page. A URL lives in config.

Warning sign: The same 8 checkout steps copied into 50 tests.

Healthy version: One OrderFacade.placeOrder() used by all 50.

KISS (keep it simple)

Pick the simplest design that solves today's problem.

In this framework: A plain function instead of a pattern when a pattern adds nothing.

Warning sign: A strategy, factory and builder for a two-line helper.

Healthy version: Add a pattern after the problem repeats.

YAGNI (you aren't gonna need it)

Do not build for imagined future needs.

In this framework: No desktop adapter until there is a desktop app to test.

Warning sign: Empty folders and interfaces "for later."

Healthy version: Add the folder when the first file needs it.

Composition over inheritance

Build objects from small parts instead of deep class trees.

In this framework: A HeaderComponent used inside pages, not a BasePageWithHeader.

Warning sign: BasePage → BaseShopPage → BaseCheckoutPage → ...

Healthy version: Shallow inheritance, shared components.

Qualities of the framework

Modularity

The framework is made of parts you can change one at a time.

In this framework: Web, mobile, desktop, API, data and reporting modules.

Warning sign: Changing the report tool requires editing tests.

Healthy version: Reporting is a listener; tests never call it.

Scalability

It still works when the suite grows from 100 to 5,000 tests.

In this framework: Parallel runs, isolated data, sharding, a factory per worker.

Warning sign: Shared accounts that force tests to run one at a time.

Healthy version: Unique data per test and a session per worker.

Maintainability

Changes are small and safe.

In this framework: A locator change edits one file.

Warning sign: A locator change edits 500 files.

Healthy version: Page objects and facades.

Extensibility

New features are added, not hacked in.

In this framework: A new platform is a new adapter and new pages. Tests stay the same.

Warning sign: if (platform == ...) in every method.

Healthy version: Factory plus one interface per screen.

Testability

Framework code can be tested in isolation.

In this framework: Constructor injection lets you pass a fake API client.

Warning sign: new HttpClient() hidden inside a method.

Healthy version: Dependencies passed in from outside.

Reusability

One implementation, many users.

In this framework: One Appium adapter used by every mobile page.

Warning sign: Each page with its own wait logic.

Healthy version: Waits written once in the adapter.

Reliability

The same code gives the same result every run.

In this framework: Waits for state, isolated data, controlled environment.

Warning sign: Hard sleeps and shared users.

Healthy version: Anti-flakiness rules enforced in review and CI.

Observability

A failure explains itself.

In this framework: Screenshots, traces, logs and versions captured on failure.

Warning sign: "Test failed" with no evidence, so people re-run it.

Healthy version: Observer listeners attach evidence automatically.

Fault tolerance

Short infrastructure hiccups do not stop the run.

In this framework: A narrow retry when a session fails to start.

Warning sign: Retrying every failed test until green.

Healthy version: Retry infrastructure, never assertions or actions.

Security

Secrets and personal data stay safe.

In this framework: Secrets come from the environment or a vault and are masked in logs.

Warning sign: const PASSWORD = "admin123" in Git.

Healthy version: Repository reads the password from an environment variable.

Reproducibility

You can recreate the exact conditions of a run.

In this framework: Pinned versions, logged random seed, fixed timezone.

Warning sign: "It passed yesterday" with no idea what changed.

Healthy version: Lock files and environment details in every report.

See it: which pattern supports which principle

Hover over or tap a row to highlight it. A dot means the pattern directly helps that principle.

Principle	POM	Factory	Singleton	Facade	Strategy	Repository	Builder	Observer	Template Method	Dependency Injection
Separation of concerns	●	●		●		●		●		
Single responsibility	●	●				●	●	●		
Loose coupling		●			●	●		●		●
High cohesion	●			●		●				
DRY	●		●	●			●		●	
Extensibility (open/closed)		●			●			●	●	
Testability		●			●	●				●
Reliability		●							●	
Observability								●		
Scalability		●				●				●

Code: single responsibility

C#

```
// BAD: one class with four jobs. Four reasons to change.
public class LoginPage
{
    public List<User> ReadUsersFromExcel() { /* data */ }
    public Task LoginAsync(User user) { /* UI */ }
    public void WriteHtmlReport() { /* reporting */ }
    public IWebDriver CreateDriver() { /* driver */ }
}

// GOOD: one job each
public sealed class LoginPage(IUiDriver ui)  { /* login screen actions only */ }
public sealed class UserRepository           { /* test users only */ }
public sealed class ReportWriter             { /* reporting only */ }
public static class DriverFactory            { /* driver creation only */ }
```

TypeScript

```
// BAD: one class with four jobs. Four reasons to change.
class LoginPage {
  readUsersFromExcel() { /* data */ }
  async login(user: User) { /* UI */ }
  writeHtmlReport() { /* reporting */ }
  createDriver() { /* driver */ }
}

// GOOD: one job each
class LoginPage      { constructor(private ui: UiDriver) {} /* login screen only */ }
class UserRepository { /* test users only */ }
class ReportWriter   { /* reporting only */ }
// createDriver() lives in framework/factory/driver-factory.ts
```

Python

```python
# BAD: one class with four jobs. Four reasons to change.
class LoginPage:
    def read_users_from_excel(self): ...    # data
    def login(self, user): ...              # UI
    def write_html_report(self): ...        # reporting
    def create_driver(self): ...            # driver


# GOOD: one job each
class LoginPage:          ...  # login screen actions only
class UserRepository:     ...  # test users only
class ReportWriter:       ...  # reporting only
# create_driver() lives in framework/factory/driver_factory.py
```

Java

```java
// BAD: one class with four jobs. Four reasons to change.
public class LoginPage {
    public List<User> readUsersFromExcel() { /* data */ }
    public void login(User user) { /* UI */ }
    public void writeHtmlReport() { /* reporting */ }
    public WebDriver createDriver() { /* driver */ }
}

// GOOD: one job each
public final class LoginPage      { /* login screen actions only */ }
public final class UserRepository { /* test users only */ }
public final class ReportWriter   { /* reporting only */ }
public final class DriverFactory  { /* driver creation only */ }
```

Code: loose coupling with dependency injection

Pass dependencies in through the constructor. Do not create them inside the class. Then the same class works with a real client, a fake client, or another environment.

C#

```csharp
// BAD: the class builds its own dependency (tight coupling)
public sealed class OrdersApi
{
    private readonly HttpClient _http =
        new() { BaseAddress = new Uri("https://staging.api.example") }; // hard-wired
}

// GOOD: the dependency comes in from outside (loose coupling)
public sealed class OrdersApi(HttpClient http)
{
    public Task<HttpResponseMessage> CreateAsync(Order order) =>
        http.PostAsJsonAsync("/orders", order);
}

// The caller decides: a real client, a fake, or another environment.
var api = new OrdersApi(new HttpClient { BaseAddress = new Uri(config.ApiBaseUrl) });
```

TypeScript

```typescript
// BAD: the class builds its own dependency (tight coupling)
class OrdersApi {
  private http = new HttpClient('https://staging.api.example'); // hard-wired
}

// GOOD: the dependency comes in from outside (loose coupling)
class OrdersApi {
  constructor(private readonly http: HttpClient) {}
  create(order: Order) {
    return this.http.post('/orders', order);
  }
}
```

```
// The caller decides: a real client, a fake, or another environment.
const api = new OrdersApi(new HttpClient(config.apiBaseUrl));
```

Python

```
# BAD: the class builds its own dependency (tight coupling)
class OrdersApi:
    def __init__(self) -> None:
        self.http = HttpClient("https://staging.api.example")  # hard-wired


# GOOD: the dependency comes in from outside (loose coupling)
class OrdersApi:
    def __init__(self, http: HttpClient) -> None:
        self.http = http

    def create(self, order: dict):
        return self.http.post("/orders", json=order)


# The caller decides: a real client, a fake, or another environment.
api = OrdersApi(HttpClient(settings.api_base_url))
```

Java

```
// BAD: the class builds its own dependency (tight coupling)
public final class OrdersApi {
    private final ApiClient http = new ApiClient("https://staging.api.example"); // hard-wired
}

// GOOD: the dependency comes in from outside (loose coupling)
public final class OrdersApi {
    private final ApiClient http;

    public OrdersApi(ApiClient http) { this.http = http; }

    public ApiResponse create(Order order) {
        return http.post("/orders", order);
    }
}

// The caller decides: a real client, a fake, or another environment.
OrdersApi api = new OrdersApi(new ApiClient(Config.get().apiBaseUrl));
```

Watch out

DRY with care. Remove repeated knowledge, not every repeated line. Two clear tests are better than one confusing shared helper.

KISS beats cleverness. If a junior tester cannot follow the code, the abstraction is too deep.

If you describe a class with "and" ("it logs in *and* reads Excel"), split it.

9 SOLID in a test framework

Five object-oriented principles, each shown with a piece of this framework.

What it is

S
Single responsibility

One reason to change. `LoginPage` changes only when the login screen changes.

O
Open / closed

Add behavior with new classes, not by editing old ones. A new SSO login is a new strategy.

L
Liskov substitution

A subtype must work wherever its parent works. Every `LoginView` can log in, on any platform.

I
Interface segregation

Keep interfaces small. Desktop apps have no URLs, so `goTo()` is not in the base driver interface.

D
Dependency inversion

Depend on interfaces. `LoginFacade` needs "a login strategy," not one specific class.

Why it matters here

SOLID is what makes the hybrid idea work. Because pages depend on a driver *interface* (D), you can plug in Playwright or Appium. Because the interface is small (I), a desktop adapter does not have to fake a "go to URL" method it cannot support. Faking it would break L.

Code: all five in one place

C#

```csharp
// S: one reason to change
public sealed class CartPage { /* cart screen only */ }

// O: add behavior with a NEW class. LoginFacade does not change.
public sealed class SsoLoginStrategy : ILoginStrategy
{
    public Task LoginAsync(User user) { /* ... */ return Task.CompletedTask; }
}

// L: any subtype can replace its parent
public static async Task SignInAsync(ILoginView view, User user)
{
    await view.OpenAsync();                    // web, mobile or desktop
    await view.LoginAsAsync(user.Email, user.Password);
}

// I: small interfaces. Desktop apps have no URL, so navigation is separate.
public interface IUiDriver   { Task ClickAsync(string target); /* ... */ }
public interface INavigable  { Task GoToAsync(string path); }
public interface IWebUiDriver : IUiDriver, INavigable { }

// D: depend on the abstraction, not the concrete class
public sealed class LoginFacade(ILoginStrategy strategy) { }
```

TypeScript

```typescript
// S: one reason to change
class CartPage { /* cart screen only */ }

// O: add behavior with a NEW class. LoginFacade does not change.
class SsoLoginStrategy implements LoginStrategy {
```

```
  async login(user: User) { /* ... */ }
}

// L: any subtype can replace its parent
async function signIn(view: LoginView, user: User) {
  await view.open();                          // web, mobile or desktop
  await view.loginAs(user.email, user.password);
}

// I: small interfaces. Desktop apps have no URL, so navigation is separate.
interface UiDriver  { click(target: string): Promise<void>; /* ... */ }
interface Navigable { goTo(path: string): Promise<void>; }
interface WebUiDriver extends UiDriver, Navigable {}

// D: depend on the abstraction, not the concrete class
class LoginFacade {
  constructor(private readonly strategy: LoginStrategy) {}
}
```

Python

```
from typing import Protocol

# S: one reason to change
class CartPage: ...  # cart screen only

# O: add behavior with a NEW class. LoginFacade does not change.
class SsoLoginStrategy:
    def login(self, user: User) -> None: ...

# L: any subtype can replace its parent
def sign_in(view: LoginView, user: User) -> None:
    view.open()                                 # web, mobile or desktop
    view.login_as(user.email, user.password)

# I: small interfaces. Desktop apps have no URL, so navigation is separate.
class UiDriver(Protocol):
    def click(self, target: str) -> None: ...

class Navigable(Protocol):
    def go_to(self, path: str) -> None: ...

# D: depend on the abstraction, not the concrete class
class LoginFacade:
    def __init__(self, strategy: LoginStrategy) -> None:
        self.strategy = strategy
```

Java

```
// S: one reason to change
public final class CartPage { /* cart screen only */ }

// O: add behavior with a NEW class. LoginFacade does not change.
public final class SsoLoginStrategy implements LoginStrategy {
    @Override public void login(User user) { /* ... */ }
}

// L: any subtype can replace its parent
static void signIn(LoginView view, User user) {
    view.open();                                // web, mobile or desktop
    view.loginAs(user.email(), user.password());
}

// I: small interfaces. Desktop apps have no URL, so navigation is separate.
public interface UiDriver  { void click(String target); /* ... */ }
public interface Navigable { void goTo(String path); }
public interface WebUiDriver extends UiDriver, Navigable {}

// D: depend on the abstraction, not the concrete class
```

```
public final class LoginFacade {
    private final LoginStrategy strategy;
    public LoginFacade(LoginStrategy strategy) { this.strategy = strategy; }
}
```

10 Configuration, environments and secrets

One place answers "where am I running, on which platform, against which URL?"

What it is

A config manager loads settings once and gives the same object to every part of the framework. That is the **Singleton** pattern. Settings come from several places. A later source overrides an earlier one:

1. Safe defaults in code
2. A non-secret environment file, such as `config/qa.env`
3. Environment variables (local `.env` or CI)
4. Command-line arguments

Why you need it

The same tests must run on QA, staging and sometimes production, on web and on devices. If URLs and platforms are written inside tests, you need a different copy of every test for each environment. With config, you change one variable.

How to do it

1. Use the same variable names in every language: `BASE_URL`, `PLATFORM`, `APPIUM_URL`, `APP_PATH`.
2. **Fail fast**: if a required value is missing, stop with a clear message before any test runs.
3. Keep secrets out of files that Git tracks. Locally, use a git-ignored `.env`. In CI, use the secret store.

Code: an environment file

config/qa.env (safe to commit: no secrets)

```
TEST_ENV=qa
PLATFORM=web                      # web | android | ios | desktop
BASE_URL=https://qa.shop.example
API_BASE_URL=https://qa.api.shop.example
APPIUM_URL=http://127.0.0.1:4723
APP_PATH=./apps/shop-qa.apk
HEADLESS=true
# Passwords come from the CI secret store or a local .env (git-ignored):
# TEST_USER_STANDARD_PASSWORD=...
```

Code: the config manager

C#

```
public enum Platform { Web, Android, Ios, Desktop }

public sealed class TestConfig
{
    // Singleton: created once, on first use, thread-safe
    private static readonly Lazy<TestConfig> Instance = new(() => new TestConfig());
    public static TestConfig Current => Instance.Value;

    public string Env { get; } = Get("TEST_ENV", "qa");
```

```
    public Platform Platform { get; } = Enum.Parse<Platform>(Get("PLATFORM", "web"), ignoreCase: true);
    public string BaseUrl { get; } = Require("BASE_URL");
    public string ApiBaseUrl { get; } = Require("API_BASE_URL");
    public Uri AppiumUrl { get; } = new(Get("APPIUM_URL", "http://127.0.0.1:4723"));
    public string AppPath { get; } = Get("APP_PATH", "");
    public bool Headless { get; } = Get("HEADLESS", "true") != "false";

    private TestConfig() { }

    private static string Get(string name, string fallback) =>
        Environment.GetEnvironmentVariable(name) ?? fallback;

    private static string Require(string name) =>
        Environment.GetEnvironmentVariable(name)
        ?? throw new InvalidOperationException($"Missing environment variable: {name}");
}
```

TypeScript

```
// framework/config/config.ts
import 'dotenv/config'; // loads a local .env file if present (git-ignored)

export type Platform = 'web' | 'android' | 'ios' | 'desktop';

function requireEnv(name: string): string {
  const value = process.env[name];
  if (!value) throw new Error(`Missing environment variable: ${name}`);
  return value;
}

class Config {
  private static instance?: Config;

  readonly env = process.env.TEST_ENV ?? 'qa';
  readonly platform = (process.env.PLATFORM ?? 'web') as Platform;
  readonly baseUrl = requireEnv('BASE_URL');
  readonly apiBaseUrl = requireEnv('API_BASE_URL');
  readonly appiumUrl = new URL(process.env.APPIUM_URL ?? 'http://127.0.0.1:4723');
  readonly appPath = process.env.APP_PATH ?? '';
  readonly headless = process.env.HEADLESS !== 'false';

  private constructor() {}

  static get(): Config {
    return (this.instance ??= new Config()); // Singleton: one shared instance
  }
}

export const config = Config.get();
```

Python

```
# framework/config/settings.py
import os
from dataclasses import dataclass, field
from functools import lru_cache


def require_env(name: str) -> str:
    value = os.getenv(name)
    if not value:
        raise RuntimeError(f"Missing environment variable: {name}")
    return value


def env(name: str, default: str):
    return field(default_factory=lambda: os.getenv(name, default))
```

```python
@dataclass(frozen=True)
class Settings:
    env: str = env("TEST_ENV", "qa")
    platform: str = env("PLATFORM", "web")
    base_url: str = field(default_factory=lambda: require_env("BASE_URL"))
    api_base_url: str = field(default_factory=lambda: require_env("API_BASE_URL"))
    appium_url: str = env("APPIUM_URL", "http://127.0.0.1:4723")
    app_path: str = env("APP_PATH", "")
    headless: bool = field(default_factory=lambda: os.getenv("HEADLESS", "true") != "false")


@lru_cache(maxsize=1)  # Singleton: the same Settings object every time
def get_settings() -> Settings:
    return Settings()
```

Java

```java
public enum Platform { WEB, ANDROID, IOS, DESKTOP }

public final class Config {
    // Singleton: one instance, created when the class loads
    private static final Config INSTANCE = new Config();
    public static Config get() { return INSTANCE; }

    public final String env        = get("TEST_ENV", "qa");
    public final Platform platform = Platform.valueOf(get("PLATFORM", "web").toUpperCase());
    public final String baseUrl    = require("BASE_URL");
    public final String apiBaseUrl = require("API_BASE_URL");
    public final String appiumUrl  = get("APPIUM_URL", "http://127.0.0.1:4723");
    public final String appPath    = get("APP_PATH", "");
    public final boolean headless  = !"false".equals(get("HEADLESS", "true"));

    private Config() {}

    private static String get(String name, String fallback) {
        return System.getenv().getOrDefault(name, fallback);
    }

    private static String require(String name) {
        String value = System.getenv(name);
        if (value == null || value.isBlank())
            throw new IllegalStateException("Missing environment variable: " + name);
        return value;
    }
}
```

Watch out

- Never commit a secret, even to a private repository. If you do by mistake, **rotate it**. Deleting the commit is not enough, because Git history keeps it.
- Keep the Singleton **read-only**. A shared object that tests can change breaks parallel runs.
- In TypeScript and Python, a module is already loaded once, so a module-level object is already a singleton. You do not need extra code for it.

11 Driver interface and driver factory

This is the heart of a hybrid framework: one small interface, one adapter per tool, one factory that picks.

What it is

A **driver interface** lists what any platform can do: click, type, read text, wait until visible, take a screenshot. An **adapter** implements it with a real tool. The Playwright adapter drives browsers. The Appium adapter drives Android, iOS and Windows apps. The **factory** reads the config and returns the right adapter.

Why you need it

- No test or page creates a driver directly. If driver creation changes (a device cloud, new capabilities), you edit one file.
- Pages depend on the interface, so the same facade and test code works on every platform.
- Each tool's quirks stay inside its adapter. Appium has no auto-wait, so its adapter adds the waits (see topic 23).

See it: pick a platform

PLATFORM	Factory returns	Settings	Runs on	Locator style	Waiting
`web`	PlaywrightDriver	`chromium.launch({ headless })`	Browser page	`[data-testid="login-button"]`	Built in (auto-wait)
`android`	AppiumUiDriver	`platformName: Android, automationName: UiAutomator2, app: APP_PATH`	Phone or emulator	`~login-button (accessibility id)`	Added by the adapter (explicit waits)
`ios`	AppiumUiDriver	`platformName: iOS, automationName: XCUITest, app: APP_PATH`	iPhone or simulator	`~login-button (accessibility id)`	Added by the adapter (explicit waits)
`desktop`	AppiumUiDriver	`platformName: Windows, automationName: Windows, app: APP_PATH`	Windows desktop app	`~LoginButton (AutomationId)`	Added by the adapter (explicit waits)

Test code is the same for every platform. It only sees `UiDriver`.

Code: the interface and the web adapter

C#

```
// framework/drivers/IUiDriver.cs: what every platform can do
public interface IUiDriver : IAsyncDisposable
{
    Task ClickAsync(string target);
    Task TypeAsync(string target, string text);
    Task<string> TextOfAsync(string target);
    Task WaitVisibleAsync(string target);
    Task<byte[]> ScreenshotAsync();
}
```

```
// Only web has URLs, so navigation is a separate, small interface (ISP)
public interface INavigable { Task GoToAsync(string path); }
public interface IWebUiDriver : IUiDriver, INavigable { }

// framework/drivers/web/PlaywrightDriver.cs: an adapter
public sealed class PlaywrightDriver(IPlaywright pw, IBrowser browser, IPage page) : IWebUiDriver
{
    // Playwright already waits for elements to be ready, so these stay thin
    public Task ClickAsync(string target) => page.Locator(target).ClickAsync();
    public Task TypeAsync(string target, string text) => page.Locator(target).FillAsync(text);
    public Task<string> TextOfAsync(string target) => page.Locator(target).InnerTextAsync();
    public Task WaitVisibleAsync(string target) =>
        page.Locator(target).WaitForAsync(new() { State = WaitForSelectorState.Visible });
    public Task<byte[]> ScreenshotAsync() => page.ScreenshotAsync();
    public async Task GoToAsync(string path) => await page.GotoAsync(path);

    public async ValueTask DisposeAsync()
    {
        await browser.CloseAsync();
        pw.Dispose();
    }
}
```

TypeScript

```
// framework/drivers/ui-driver.ts: what every platform can do
export interface UiDriver {
  click(target: string): Promise<void>;
  type(target: string, text: string): Promise<void>;
  textOf(target: string): Promise<string>;
  waitVisible(target: string): Promise<void>;
  screenshot(): Promise<Buffer>;
  quit(): Promise<void>;
}

// Only web has URLs, so navigation is a separate, small interface (ISP)
export interface Navigable { goTo(path: string): Promise<void>; }
export interface WebUiDriver extends UiDriver, Navigable {}

// framework/drivers/web/playwright-driver.ts: an adapter
import { Browser, Page } from '@playwright/test';

export class PlaywrightDriver implements WebUiDriver {
  constructor(private readonly browser: Browser, private readonly page: Page) {}

  // Playwright already waits for elements to be ready, so these stay thin
  click(target: string) { return this.page.locator(target).click(); }
  type(target: string, text: string) { return this.page.locator(target).fill(text); }
  textOf(target: string) { return this.page.locator(target).innerText(); }
  waitVisible(target: string) { return this.page.locator(target).waitFor({ state: 'visible' }); }
  screenshot() { return this.page.screenshot(); }
  async goTo(path: string) { await this.page.goto(path); }
  quit() { return this.browser.close(); }
}
```

Python

```
# framework/drivers/ui_driver.py: what every platform can do
from typing import Protocol


class UiDriver(Protocol):
    def click(self, target: str) -> None: ...
    def type(self, target: str, text: str) -> None: ...
    def text_of(self, target: str) -> str: ...
    def wait_visible(self, target: str) -> None: ...
    def screenshot(self) -> bytes: ...
    def quit(self) -> None: ...
```

```python
# Only web has URLs, so navigation is a separate, small interface (ISP)
class Navigable(Protocol):
    def go_to(self, path: str) -> None: ...


# framework/drivers/web/playwright_driver.py: an adapter
class PlaywrightDriver:
    def __init__(self, pw, browser, page) -> None:
        self.pw, self.browser, self.page = pw, browser, page

    # Playwright already waits for elements to be ready, so these stay thin
    def click(self, target: str) -> None: self.page.locator(target).click()
    def type(self, target: str, text: str) -> None: self.page.locator(target).fill(text)
    def text_of(self, target: str) -> str: return self.page.locator(target).inner_text()
    def wait_visible(self, target: str) -> None: self.page.locator(target).wait_for(state="visible")
    def screenshot(self) -> bytes: return self.page.screenshot()
    def go_to(self, path: str) -> None: self.page.goto(path)

    def quit(self) -> None:
        self.browser.close()
        self.pw.stop()
```

Java

```java
// framework/drivers/UiDriver.java: what every platform can do
public interface UiDriver extends AutoCloseable {
    void click(String target);
    void type(String target, String text);
    String textOf(String target);
    void waitVisible(String target);
    byte[] screenshot();
}

// Only web has URLs, so navigation is a separate, small interface (ISP)
public interface Navigable { void goTo(String path); }
public interface WebUiDriver extends UiDriver, Navigable {}

// framework/drivers/web/PlaywrightDriver.java: an adapter
public final class PlaywrightDriver implements WebUiDriver {
    private final Playwright pw;
    private final Browser browser;
    private final Page page;

    PlaywrightDriver(Playwright pw, Browser browser, Page page) {
        this.pw = pw; this.browser = browser; this.page = page;
    }

    // Playwright already waits for elements to be ready, so these stay thin
    public void click(String t)              { page.locator(t).click(); }
    public void type(String t, String text)  { page.locator(t).fill(text); }
    public String textOf(String t)           { return page.locator(t).innerText(); }
    public void waitVisible(String t)        { page.locator(t).waitFor(); } // default state: visible
    public byte[] screenshot()               { return page.screenshot(); }
    public void goTo(String path)            { page.navigate(path); }
    public void close()                      { browser.close(); pw.close(); }
}
```

Code: the factory

C#

```csharp
using OpenQA.Selenium.Appium;
using OpenQA.Selenium.Appium.Android;
using OpenQA.Selenium.Appium.iOS;
using OpenQA.Selenium.Appium.Windows;
```

```
public static class DriverFactory
{
    public static async Task<IUiDriver> CreateAsync(TestConfig c) => c.Platform switch
    {
        Platform.Web     => await PlaywrightDriverLauncher.LaunchAsync(c),
        Platform.Android => new AppiumUiDriver(new AndroidDriver(c.AppiumUrl, Options("Android",
"UiAutomator2", c))),
        Platform.Ios     => new AppiumUiDriver(new IOSDriver(c.AppiumUrl, Options("iOS", "XCUITest", c))),
        Platform.Desktop => new AppiumUiDriver(new WindowsDriver(c.AppiumUrl, Options("Windows", "Windows",
c))),
        _ => throw new NotSupportedException($"Unsupported platform: {c.Platform}")
    };

    private static AppiumOptions Options(string platform, string automation, TestConfig c) => new()
    {
        PlatformName = platform,
        AutomationName = automation,
        App = c.AppPath
    };
}

public static class PlaywrightDriverLauncher
{
    public static async Task<IUiDriver> LaunchAsync(TestConfig c)
    {
        var pw = await Playwright.CreateAsync();
        var browser = await pw.Chromium.LaunchAsync(new() { Headless = c.Headless });
        var page = await browser.NewPageAsync(new() { BaseURL = c.BaseUrl });
        return new PlaywrightDriver(pw, browser, page);
    }
}
```

TypeScript

```
// framework/factory/driver-factory.ts
import { chromium } from '@playwright/test';
import { remote } from 'webdriverio';

const appium = { hostname: config.appiumUrl.hostname, port: Number(config.appiumUrl.port) };

export async function createDriver(platform: Platform): Promise<UiDriver> {
  switch (platform) {
    case 'web': {
      const browser = await chromium.launch({ headless: config.headless });
      const page = await browser.newPage({ baseURL: config.baseUrl });
      return new PlaywrightDriver(browser, page);
    }
    case 'android':
      return new AppiumDriver(await remote({ ...appium, capabilities: caps('Android', 'UiAutomator2') }));
    case 'ios':
      return new AppiumDriver(await remote({ ...appium, capabilities: caps('iOS', 'XCUITest') }));
    case 'desktop':
      return new AppiumDriver(await remote({ ...appium, capabilities: caps('Windows', 'Windows') }));
    default:
      throw new Error(`Unsupported platform: ${platform}`);
  }
}

function caps(platformName: string, automationName: string) {
  return {
    platformName,
    'appium:automationName': automationName,
    'appium:app': config.appPath,
  };
}
```

Python

```python
# framework/factory/driver_factory.py
from appium import webdriver as appium
from appium.options.android import UiAutomator2Options
from appium.options.ios import XCUITestOptions
from appium.options.windows import WindowsOptions
from playwright.sync_api import sync_playwright

APPIUM_OPTIONS = {
    "android": UiAutomator2Options,
    "ios": XCUITestOptions,
    "desktop": WindowsOptions,
}


def create_driver(settings: Settings) -> UiDriver:
    if settings.platform == "web":
        pw = sync_playwright().start()
        browser = pw.chromium.launch(headless=settings.headless)
        page = browser.new_page(base_url=settings.base_url)
        return PlaywrightDriver(pw, browser, page)

    if settings.platform not in APPIUM_OPTIONS:
        raise ValueError(f"Unsupported platform: {settings.platform}")

    options = APPIUM_OPTIONS[settings.platform]()
    options.app = settings.app_path
    return AppiumDriver(appium.Remote(settings.appium_url, options=options))
```

Java

```java
import io.appium.java_client.android.AndroidDriver;
import io.appium.java_client.android.options.UiAutomator2Options;
import io.appium.java_client.ios.IOSDriver;
import io.appium.java_client.ios.options.XCUITestOptions;
import io.appium.java_client.windows.WindowsDriver;
import io.appium.java_client.windows.options.WindowsOptions;

public final class DriverFactory {
    private DriverFactory() {}

    public static UiDriver create(Config c) throws Exception {
        URL appium = URI.create(c.appiumUrl).toURL();
        return switch (c.platform) {
            case WEB     -> PlaywrightDriverLauncher.launch(c);
            case ANDROID -> new AppiumUiDriver(new AndroidDriver(appium, new
UiAutomator2Options().setApp(c.appPath)));
            case IOS     -> new AppiumUiDriver(new IOSDriver(appium, new XCUITestOptions().setApp(c.appPath)));
            case DESKTOP -> new AppiumUiDriver(new WindowsDriver(appium, new
WindowsOptions().setApp(c.appPath)));
        };
    }
}

final class PlaywrightDriverLauncher {
    static UiDriver launch(Config c) {
        Playwright pw = Playwright.create();
        Browser browser = pw.chromium().launch(new BrowserType.LaunchOptions().setHeadless(c.headless));
        Page page = browser.newPage(new Browser.NewPageOptions().setBaseURL(c.baseUrl));
        return new PlaywrightDriver(pw, browser, page);
    }
}
```

Watch out

- For web-only suites, your runner already manages the browser (Playwright Test's `page`, pytest-playwright's `page`, NUnit's `PageTest`). Use the runner's lifecycle there. Use your own factory when you need several platforms or tools behind one interface.
- Mobile parallel runs need one device or emulator per worker, plus unique ports (`systemPort` for Android, `wdaLocalPort` for iOS).
- Never store the driver in a static or global field. Parallel tests would share it.

12 Base page and the Template Method pattern

The base page fixes the steps every page follows. Each page fills in only its own details.

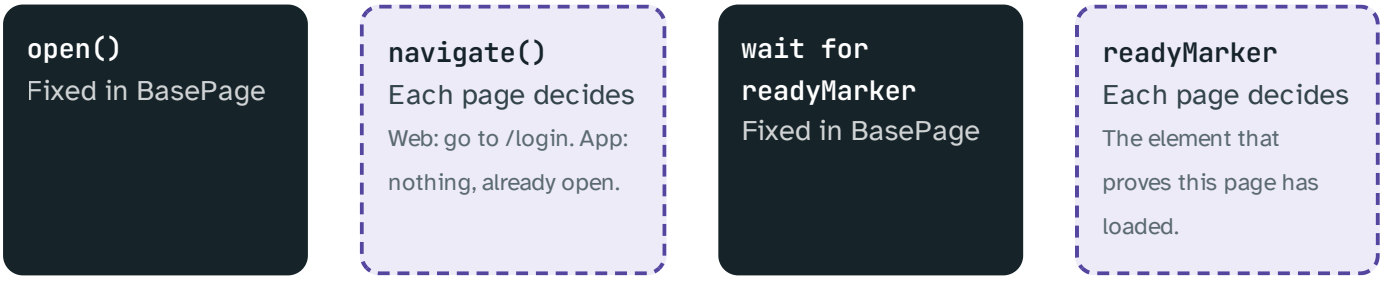
What it is

Template Method means a parent class defines the order of steps, and child classes fill in some of the steps. Here, "open a page" is always: navigate, then wait until the page is ready. A web page navigates by URL. A mobile screen may already be open. Both wait for their own "ready marker."

Why you need it

Without it, some pages wait after opening and some do not. Tests then fail randomly on slow machines. With a template, "open, then wait until ready" happens the same way everywhere, and nobody can forget the wait.

See it



Code

C#

```
public abstract class BasePage(IUiDriver ui)
{
    protected IUiDriver Ui { get; } = ui;

    // Template method: the steps and their order are fixed here.
    // Each page only fills in the details.
    public async Task OpenAsync()
    {
        await NavigateAsync();
        await Ui.WaitVisibleAsync(ReadyMarker);   // proves the page is ready
    }

    protected abstract Task NavigateAsync();     // how to arrive on this page
    protected abstract string ReadyMarker { get; } // element that shows it loaded
}
```

TypeScript

```
export abstract class BasePage {
  constructor(protected readonly ui: UiDriver) {}

  // Template method: the steps and their order are fixed here.
  // Each page only fills in the details.
  async open(): Promise<this> {
    await this.navigate();
    await this.ui.waitVisible(this.readyMarker); // proves the page is ready
    return this;
  }

  protected abstract navigate(): Promise<void>;   // how to arrive on this page
  protected abstract readonly readyMarker: string; // element that shows it loaded
}
```

Python

```python
from abc import ABC, abstractmethod


class BasePage(ABC):
    def __init__(self, ui: UiDriver) -> None:
        self.ui = ui

    # Template method: the steps and their order are fixed here.
    # Each page only fills in the details.
    def open(self) -> "BasePage":
        self.navigate()
        self.ui.wait_visible(self.ready_marker)  # proves the page is ready
        return self

    @abstractmethod
    def navigate(self) -> None: ...              # how to arrive on this page

    @property
    @abstractmethod
    def ready_marker(self) -> str: ...           # element that shows it loaded
```

Java

```java
public abstract class BasePage {
    protected final UiDriver ui;

    protected BasePage(UiDriver ui) { this.ui = ui; }

    // Template method: the steps and their order are fixed here.
    // "final" stops subclasses from changing the order.
    public final void open() {
        navigate();
        ui.waitVisible(readyMarker());   // proves the page is ready
    }

    protected abstract void navigate();       // how to arrive on this page
    protected abstract String readyMarker();  // element that shows it loaded
}
```

Watch out

! Keep the base page small. When it grows to 50 helper methods, it becomes a "god class." Put general helpers in `utils/` and waits in the driver adapter.

! Prefer composition for shared widgets. A header that appears on many pages is its own small component object, not more base-class methods.

13 Page Object Model (POM) across platforms

A page object is the only place that knows a screen's locators. Tests talk to it in plain actions.

What it is

Each screen gets a class. The class holds the screen's locators and offers actions like `loginAs(email, password)`. In a hybrid framework, each platform has its own page class, and all of them implement one shared interface such as `LoginView`. A small **page factory** picks the right class, so tests never choose it themselves.

Why you need it

When a locator changes, you fix it in one file instead of in every test that touches that screen. Try the slider to see the difference.

See it: a login button's locator changes

Tests that use the login screen	Edits without POM	Edits with POM
1	1	1
10	10	1
120	120	1
500	500	1

Without POM, the locator is copied into every test, and missing one leaves a broken test. With POM, you change one line in `WebLoginPage`.

How to do it

1. One class per screen, per platform: `WebLoginPage`, `NativeLoginScreen`.
2. Locators are private. Only actions and simple state checks are public.
3. Actions describe user intent: `loginAs()`, not `clickButton3()`.
4. Do not put business assertions in pages. A page may offer `expectLoaded()`. The test decides what "correct" means.

Code: one interface, one class per platform

In TypeScript the native locators start with `~`, which is WebdriverIO's short form for accessibility id. In the other languages, the Appium adapter treats the target as an accessibility id. On Windows, accessibility id means the control's AutomationId.

C#

```
// What a login screen can do, on any platform
public interface ILoginView
{
    Task OpenAsync();
    Task LoginAsAsync(string email, string password);
}

// Web: CSS selectors on data-testid attributes
public sealed class WebLoginPage(IWebUiDriver web) : BasePage(web), ILoginView
{
```

```
    protected override string ReadyMarker => "[data-testid='login-form']";
    protected override Task NavigateAsync() => web.GoToAsync("/login");

    public async Task LoginAsAsync(string email, string password)
    {
        await Ui.TypeAsync("[data-testid='email']", email);
        await Ui.TypeAsync("[data-testid='password']", password);
        await Ui.ClickAsync("[data-testid='login-button']");
    }
}

// Mobile and desktop: accessibility id (AutomationId on Windows)
public sealed class NativeLoginScreen(IUiDriver ui) : BasePage(ui), ILoginView
{
    protected override string ReadyMarker => "login-form";
    protected override Task NavigateAsync() => Task.CompletedTask; // app opens here

    public async Task LoginAsAsync(string email, string password)
    {
        await Ui.TypeAsync("email", email);
        await Ui.TypeAsync("password", password);
        await Ui.ClickAsync("login-button");
    }
}

// Page factory: tests never pick the class themselves
public static class Pages
{
    public static ILoginView Login(IUiDriver ui) => ui is IWebUiDriver web
        ? new WebLoginPage(web)
        : new NativeLoginScreen(ui);
}
```

TypeScript

```
// What a login screen can do, on any platform
export interface LoginView {
  open(): Promise<unknown>;
  loginAs(email: string, password: string): Promise<void>;
}

// Web: CSS selectors on data-testid attributes
export class WebLoginPage extends BasePage implements LoginView {
  protected readonly readyMarker = '[data-testid="login-form"]';
  constructor(private readonly web: WebUiDriver) { super(web); }
  protected navigate() { return this.web.goTo('/login'); }

  async loginAs(email: string, password: string) {
    await this.ui.type('[data-testid="email"]', email);
    await this.ui.type('[data-testid="password"]', password);
    await this.ui.click('[data-testid="login-button"]');
  }
}

// Mobile and desktop: "~" means accessibility id (AutomationId on Windows)
export class NativeLoginScreen extends BasePage implements LoginView {
  protected readonly readyMarker = '~login-form';
  protected async navigate() { /* the app opens on this screen */ }

  async loginAs(email: string, password: string) {
    await this.ui.type('~email', email);
    await this.ui.type('~password', password);
    await this.ui.click('~login-button');
  }
}

// Page factory: tests never pick the class themselves
export const pages = {
```

```
  login: (ui: UiDriver): LoginView ⇒
    'goTo' in ui ? new WebLoginPage(ui as WebUiDriver) : new NativeLoginScreen(ui),
};
```

Python

```python
# What a login screen can do, on any platform
class LoginView(Protocol):
    def open(self) -> object: ...
    def login_as(self, email: str, password: str) -> None: ...


# Web: CSS selectors on data-testid attributes
class WebLoginPage(BasePage):
    ready_marker = "[data-testid='login-form']"

    def navigate(self) -> None:
        self.ui.go_to("/login")

    def login_as(self, email: str, password: str) -> None:
        self.ui.type("[data-testid='email']", email)
        self.ui.type("[data-testid='password']", password)
        self.ui.click("[data-testid='login-button']")


# Mobile and desktop: accessibility id (AutomationId on Windows)
class NativeLoginScreen(BasePage):
    ready_marker = "login-form"

    def navigate(self) -> None:
        pass  # the app opens on this screen

    def login_as(self, email: str, password: str) -> None:
        self.ui.type("email", email)
        self.ui.type("password", password)
        self.ui.click("login-button")


# Page factory: tests never pick the class themselves
def login_view(ui: UiDriver) -> LoginView:
    return WebLoginPage(ui) if hasattr(ui, "go_to") else NativeLoginScreen(ui)
```

Java

```java
// What a login screen can do, on any platform
public interface LoginView {
    void open();
    void loginAs(String email, String password);
}

// Web: CSS selectors on data-testid attributes
public final class WebLoginPage extends BasePage implements LoginView {
    private final WebUiDriver web;
    public WebLoginPage(WebUiDriver web) { super(web); this.web = web; }

    @Override protected String readyMarker() { return "[data-testid='login-form']"; }
    @Override protected void navigate() { web.goTo("/login"); }

    @Override public void loginAs(String email, String password) {
        ui.type("[data-testid='email']", email);
        ui.type("[data-testid='password']", password);
        ui.click("[data-testid='login-button']");
    }
}

// Mobile and desktop: accessibility id (AutomationId on Windows)
public final class NativeLoginScreen extends BasePage implements LoginView {
    public NativeLoginScreen(UiDriver ui) { super(ui); }
```

```
    @Override protected String readyMarker() { return "login-form"; }
    @Override protected void navigate() { /* the app opens on this screen */ }

    @Override public void loginAs(String email, String password) {
        ui.type("email", email);
        ui.type("password", password);
        ui.click("login-button");
    }
}

// Page factory: tests never pick the class themselves
public final class Pages {
    public static LoginView login(UiDriver ui) {
        return ui instanceof WebUiDriver web ? new WebLoginPage(web) : new NativeLoginScreen(ui);
    }
}
```

Watch out

> **!** Do not write empty wrappers that only rename one built-in call. `clickLoginButton()` adds nothing over `loginAs()`.

> **!** Mobile and desktop screens often share accessibility ids with each other. If they do, one native class can serve both, as above. If they do not, split them.

14 Facade: business flows in one call

A facade turns many screen steps into one business action, like "place an order."

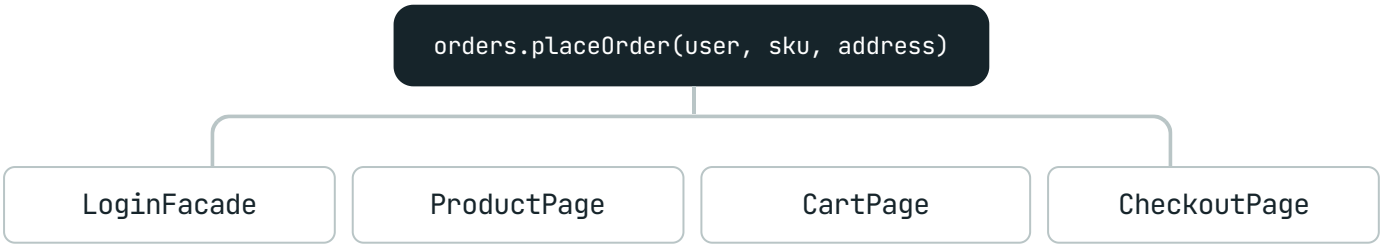
What it is

A facade sits between tests and pages. It calls several pages, and sometimes API clients, in the right order. The test calls one method and reads like a sentence.

Why you need it

Fifty checkout tests should not each repeat the same eight steps. When the checkout flow gains a new step, you add it once, in the facade.

See it



Code

C#

```
// One business action that spans several screens
public sealed class OrderFacade(LoginFacade login, ProductPage product, CartPage cart, CheckoutPage checkout)
{
    public async Task<string> PlaceOrderAsync(User user, string sku, Address address)
    {
        await login.LoginAsAsync(user);
        await product.OpenAsync(sku);
        await product.AddToCartAsync();
        await cart.ProceedAsync();
        await checkout.EnterShippingAsync(address);
        await checkout.SubmitAsync();
        return await checkout.OrderNumberAsync();
    }
}

// The test reads like a sentence
var orderNumber = await orders.PlaceOrderAsync(user, "SKU-MOUSE-01", address);
```

TypeScript

```
// One business action that spans several screens
export class OrderFacade {
  constructor(
    private readonly login: LoginFacade,
    private readonly product: ProductPage,
    private readonly cart: CartPage,
    private readonly checkout: CheckoutPage,
  ) {}

  async placeOrder(user: User, sku: string, address: Address): Promise<string> {
    await this.login.loginAs(user);
    await this.product.open(sku);
    await this.product.addToCart();
    await this.cart.proceed();
    await this.checkout.enterShipping(address);
    await this.checkout.submit();
```

```
    return this.checkout.orderNumber();
  }
}

// The test reads like a sentence
const orderNumber = await orders.placeOrder(user, 'SKU-MOUSE-01', address);
```

Python

```
# One business action that spans several screens
class OrderFacade:
    def __init__(self, login, product, cart, checkout) -> None:
        self.login, self.product = login, product
        self.cart, self.checkout = cart, checkout

    def place_order(self, user: User, sku: str, address: Address) -> str:
        self.login.login_as(user)
        self.product.open(sku)
        self.product.add_to_cart()
        self.cart.proceed()
        self.checkout.enter_shipping(address)
        self.checkout.submit()
        return self.checkout.order_number()


# The test reads like a sentence
order_number = orders.place_order(user, "SKU-MOUSE-01", address)
```

Java

```
// One business action that spans several screens
public final class OrderFacade {
    private final LoginFacade login;
    private final ProductPage product;
    private final CartPage cart;
    private final CheckoutPage checkout;

    public OrderFacade(LoginFacade login, ProductPage product, CartPage cart, CheckoutPage checkout) {
        this.login = login; this.product = product; this.cart = cart; this.checkout = checkout;
    }

    public String placeOrder(User user, String sku, Address address) {
        login.loginAs(user);
        product.open(sku);
        product.addToCart();
        cart.proceed();
        checkout.enterShipping(address);
        checkout.submit();
        return checkout.orderNumber();
    }
}

// The test reads like a sentence
String orderNumber = orders.placeOrder(user, "SKU-MOUSE-01", address);
```

Watch out

! No facade for a single click. A facade earns its place when it joins two or more steps that always go together.

! Keep the business check in the test: `expect(orderNumber).toMatch(...)`. Do not hide it inside the facade.

15 Strategy: swap behavior at runtime

When there are several ways to do the same job, each way becomes its own class with the same interface.

What it is

A strategy is one interchangeable way of doing something. Login is a good example. You can log in through the screen (UI), through an API token (fast setup), with an OTP, or with SSO. The facade holds "a login strategy" and does not care which one.

Why you need it

Only the login tests need to log in through the screen. The other 300 tests only need a logged-in user. An API session strategy skips the login screen, so those tests are faster and do not fail because of a login screen problem.

See it: switch the strategy

The test calls `loginFacade.loginAs(user)` in both cases. Only the injected strategy changes.

UiLoginStrategy

1. Open /login
2. Wait for the login form
3. Type the email
4. Type the password
5. Click "Log in"
6. Wait for the dashboard

Six screen steps. Use it in the login tests, where the screen itself is what you are testing.

ApiSessionStrategy

1. POST /auth/token with the user's credentials
2. Add the session cookie to the browser context
3. Open the page the test needs

No login screen at all. Use it in the other tests: they are faster, and a login screen bug cannot make them fail.

Code

```C#
public interface ILoginStrategy { Task LoginAsync(User user); }

// Through the screen: use when the login itself is under test
public sealed class UiLoginStrategy(ILoginView view) : ILoginStrategy
{
    public async Task LoginAsync(User u)
    {
        await view.OpenAsync();
        await view.LoginAsAsync(u.Email, u.Password);
    }
}

// Through the API: fast setup when login is NOT what you are testing
public sealed class ApiSessionStrategy(AuthApi auth, IBrowserContext context) : ILoginStrategy
{
    public async Task LoginAsync(User u)
    {
        var token = await auth.GetTokenAsync(u.Email, u.Password);
        await context.AddCookiesAsync([new() { Name = "session", Value = token, Url =
TestConfig.Current.BaseUrl }]);
```

```
    }
}

// The facade does not know which strategy it has
public sealed class LoginFacade(ILoginStrategy strategy)
{
    public Task LoginAsAsync(User user) => strategy.LoginAsync(user);
}
```

TypeScript

```
export interface LoginStrategy { login(user: User): Promise<void>; }

// Through the screen: use when the login itself is under test
export class UiLoginStrategy implements LoginStrategy {
  constructor(private readonly view: LoginView) {}
  async login(u: User) {
    await this.view.open();
    await this.view.loginAs(u.email, u.password);
  }
}

// Through the API: fast setup when login is NOT what you are testing
export class ApiSessionStrategy implements LoginStrategy {
  constructor(private readonly auth: AuthApi, private readonly context: BrowserContext) {}
  async login(u: User) {
    const token = await this.auth.getToken(u.email, u.password);
    await this.context.addCookies([{ name: 'session', value: token, url: config.baseUrl }]);
  }
}

// The facade does not know which strategy it has
export class LoginFacade {
  constructor(private readonly strategy: LoginStrategy) {}
  loginAs(user: User) { return this.strategy.login(user); }
}
```

Python

```
class LoginStrategy(Protocol):
    def login(self, user: User) -> None: ...


# Through the screen: use when the login itself is under test
class UiLoginStrategy:
    def __init__(self, view: LoginView) -> None:
        self.view = view

    def login(self, u: User) -> None:
        self.view.open()
        self.view.login_as(u.email, u.password)


# Through the API: fast setup when login is NOT what you are testing
class ApiSessionStrategy:
    def __init__(self, auth: AuthApi, context) -> None:
        self.auth, self.context = auth, context

    def login(self, u: User) -> None:
        token = self.auth.get_token(u.email, u.password)
        self.context.add_cookies([{"name": "session", "value": token, "url": get_settings().base_url}])


# The facade does not know which strategy it has
class LoginFacade:
    def __init__(self, strategy: LoginStrategy) -> None:
        self.strategy = strategy
```

```
    def login_as(self, user: User) -> None:
        self.strategy.login(user)
```

Java

```
public interface LoginStrategy { void login(User user); }

// Through the screen: use when the login itself is under test
public final class UiLoginStrategy implements LoginStrategy {
    private final LoginView view;
    public UiLoginStrategy(LoginView view) { this.view = view; }

    @Override public void login(User u) {
        view.open();
        view.loginAs(u.email(), u.password());
    }
}

// Through the API: fast setup when login is NOT what you are testing
public final class ApiSessionStrategy implements LoginStrategy {
    private final AuthApi auth;
    private final BrowserContext context;
    public ApiSessionStrategy(AuthApi auth, BrowserContext context) { this.auth = auth; this.context = context;
}

    @Override public void login(User u) {
        String token = auth.getToken(u.email(), u.password());
        context.addCookies(List.of(new Cookie("session", token).setUrl(Config.get().baseUrl())));
    }
}

// The facade does not know which strategy it has
public final class LoginFacade {
    private final LoginStrategy strategy;
    public LoginFacade(LoginStrategy strategy) { this.strategy = strategy; }
    public void loginAs(User user) { strategy.login(user); }
}
```

Watch out

Keep axes separate. Login method (password, OTP, SSO) is one strategy axis. Platform (web, Android, iOS, desktop) is handled by the driver and page layers. Do not create `AndroidOtpLoginStrategy`, one class for every combination.

Do not replace a small, stable `if` with a strategy just to use a pattern.

16 Test data: JSON, Excel, CSV, database, API

Tests ask for data by meaning ("an admin user"). A repository decides where it comes from.

What it is

Three patterns work together here:

Repository The only code that reads test data. Tests call `users.get("admin")`, never "row 4 of sheet 2."

Source A small reader for one format: JSON, Excel, CSV, a database or an API. The repository uses a source and hides it.

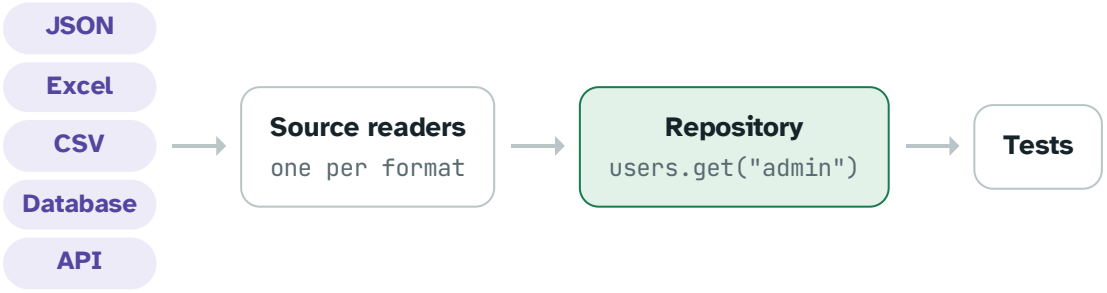
Builder Creates complex objects with safe defaults, so each test sets only the fields it cares about.

Data-driven testing runs one test body many times, once for each row of data.

Why you need it

Shared, hard-coded data is one of the biggest causes of flaky tests. Test A changes the account that test B needs. With a repository, you can move from Excel to JSON, or from JSON to an API, without touching a single test.

See it: where data comes from



Source	Good for	Watch out for
JSON or YAML in Git	Static reference data, users, settings	Never put secrets in these files
CSV in Git	Simple data-driven tables	Poor for nested data
Excel	Data that business users maintain	Hard to review in pull requests, and merge conflicts cannot be fixed by hand
Database	Large or changing data	Needs isolation and cleanup
API	Fresh data for each test	Depends on the API being stable

Good default: JSON for static reference data, the API for per-test data. If business people own an Excel file, read it through the repository, or convert it to JSON in a build step so changes show up in pull requests.

Code: JSON user data (no passwords inside)

```
testdata/json/users.json (no passwords, only the name of the secret)
{
  "admin":    { "email": "admin@shop.example",    "role": "admin",    "passwordVar": "TEST_USER_ADMIN_PASSWORD"
},
```

```
  "standard": { "email": "standard@shop.example", "role": "customer", "passwordVar":
"TEST_USER_STANDARD_PASSWORD" },
  "blocked":  { "email": "blocked@shop.example",  "role": "customer", "passwordVar":
"TEST_USER_BLOCKED_PASSWORD" }
}
```

Code: repository with a JSON source

The JSON holds only the *name* of the environment variable that stores the password. The repository reads the real value from the environment.

C#

```
public sealed record UserRecord(string Email, string Role, string PasswordVar);
public sealed record User(string Email, string Role, string Password);

// Where the data comes from is hidden behind this interface
public interface IUserSource { IReadOnlyDictionary<string, UserRecord> Load(); }

public sealed class JsonUserSource(string path) : IUserSource
{
    public IReadOnlyDictionary<string, UserRecord> Load() =>
        JsonSerializer.Deserialize<Dictionary<string, UserRecord>>(
            File.ReadAllText(path), new JsonSerializerOptions(JsonSerializerDefaults.Web))!;
}

// The repository is the ONLY code that reads user data
public sealed class UserRepository(IUserSource source)
{
    private readonly IReadOnlyDictionary<string, UserRecord> _users = source.Load();

    public User Get(string key)
    {
        if (!_users.TryGetValue(key, out var r))
            throw new KeyNotFoundException($"Missing test user: {key}");

        var password = Environment.GetEnvironmentVariable(r.PasswordVar)
            ?? throw new InvalidOperationException($"Missing secret: {r.PasswordVar}");
        return new User(r.Email, r.Role, password);
    }
}

// Usage: tests ask by meaning, not by file or row number
var users = new UserRepository(new JsonUserSource("testdata/json/users.json"));
var admin = users.Get("admin");
```

TypeScript

```
import { readFileSync } from 'node:fs';

type UserRecord = { email: string; role: string; passwordVar: string };
export type User = { email: string; role: string; password: string };

// Where the data comes from is hidden behind this interface
export interface UserSource { load(): Record<string, UserRecord>; }

export class JsonUserSource implements UserSource {
  constructor(private readonly path: string) {}
  load() { return JSON.parse(readFileSync(this.path, 'utf-8')); }
}

// The repository is the ONLY code that reads user data
export class UserRepository {
  private readonly users: Record<string, UserRecord>;
  constructor(source: UserSource) { this.users = source.load(); }

  get(key: string): User {
```

```
    const r = this.users[key];
    if (!r) throw new Error(`Missing test user: ${key}`);
    const password = process.env[r.passwordVar];
    if (!password) throw new Error(`Missing secret: ${r.passwordVar}`);
    return { email: r.email, role: r.role, password };
  }
}

// Usage: tests ask by meaning, not by file or row number
const users = new UserRepository(new JsonUserSource('testdata/json/users.json'));
const admin = users.get('admin');
```

Python

```
import json
import os
from dataclasses import dataclass
from pathlib import Path
from typing import Protocol


@dataclass(frozen=True)
class User:
    email: str
    role: str
    password: str


# Where the data comes from is hidden behind this interface
class UserSource(Protocol):
    def load(self) -> dict[str, dict]: ...


class JsonUserSource:
    def __init__(self, path: str) -> None:
        self.path = Path(path)

    def load(self) -> dict[str, dict]:
        return json.loads(self.path.read_text(encoding="utf-8"))


# The repository is the ONLY code that reads user data
class UserRepository:
    def __init__(self, source: UserSource) -> None:
        self.users = source.load()

    def get(self, key: str) -> User:
        if key not in self.users:
            raise KeyError(f"Missing test user: {key}")
        r = self.users[key]
        password = os.getenv(r["passwordVar"])
        if not password:
            raise RuntimeError(f"Missing secret: {r['passwordVar']}")
        return User(r["email"], r["role"], password)


# Usage: tests ask by meaning, not by file or row number
users = UserRepository(JsonUserSource("testdata/json/users.json"))
admin = users.get("admin")
```

Java

```
public record UserRecord(String email, String role, String passwordVar) {}
public record User(String email, String role, String password) {}

// Where the data comes from is hidden behind this interface
public interface UserSource { Map<String, UserRecord> load(); }

public final class JsonUserSource implements UserSource {
```

```
    private final Path path;
    public JsonUserSource(String path) { this.path = Path.of(path); }

    @Override public Map<String, UserRecord> load() {
        try {
            return new ObjectMapper().readValue(path.toFile(), new TypeReference<>() {});
        } catch (IOException e) {
            throw new UncheckedIOException("Cannot read " + path, e);
        }
    }
}

// The repository is the ONLY code that reads user data
public final class UserRepository {
    private final Map<String, UserRecord> users;
    public UserRepository(UserSource source) { this.users = source.load(); }

    public User get(String key) {
        UserRecord r = users.get(key);
        if (r == null) throw new NoSuchElementException("Missing test user: " + key);
        String password = System.getenv(r.passwordVar());
        if (password == null) throw new IllegalStateException("Missing secret: " + r.passwordVar());
        return new User(r.email(), r.role(), password);
    }
}

// Usage: tests ask by meaning, not by file or row number
UserRepository users = new UserRepository(new JsonUserSource("testdata/json/users.json"));
User admin = users.get("admin");
```

Code: reading Excel

C#

```
// NuGet: ClosedXML
using ClosedXML.Excel;

public static class ExcelReader
{
    // Row 1 = column names. Every other row becomes a dictionary.
    public static List<Dictionary<string, string>> ReadSheet(string path, string sheet)
    {
        using var workbook = new XLWorkbook(path);
        var ws = workbook.Worksheet(sheet);
        var headers = ws.FirstRowUsed()!.CellsUsed().Select(c => c.GetString()).ToList();

        return ws.RowsUsed().Skip(1)
            .Select(row => headers
                .Select((name, i) => (name, value: row.Cell(i + 1).GetString()))
                .ToDictionary(x => x.name, x => x.value))
            .ToList();
    }
}
```

TypeScript

```
// npm: exceljs
import ExcelJS from 'exceljs';

// Row 1 = column names. Every other row becomes an object.
export async function readSheet(path: string, sheet: string): Promise<Record<string, string>[]> {
  const workbook = new ExcelJS.Workbook();
  await workbook.xlsx.readFile(path);
  const ws = workbook.getWorksheet(sheet);
  if (!ws) throw new Error(`Sheet not found: ${sheet}`);

  const headers = (ws.getRow(1).values as unknown[]).slice(1).map(String);
  const rows: Record<string, string>[] = [];
```

```
  ws.eachRow((row, rowNumber) => {
    if (rowNumber === 1) return;
    const values = (row.values as unknown[]).slice(1);
    rows.push(Object.fromEntries(headers.map((h, i) => [h, String(values[i] ?? '')])));
  });
  return rows;
}
```

Python

```
# pip: openpyxl
from openpyxl import load_workbook


def read_sheet(path: str, sheet: str) -> list[dict[str, str]]:
    """Row 1 = column names. Every other row becomes a dict."""
    workbook = load_workbook(path, read_only=True, data_only=True)
    try:
        rows = workbook[sheet].iter_rows(values_only=True)
        headers = [str(h) for h in next(rows)]
        return [
            {h: "" if v is None else str(v) for h, v in zip(headers, row)}
            for row in rows
        ]
    finally:
        workbook.close()
```

Java

```
// Maven: org.apache.poi:poi-ooxml
public final class ExcelReader {
    // Row 1 = column names. Every other row becomes a map.
    public static List<Map<String, String>> readSheet(Path path, String sheet) throws IOException {
        try (InputStream in = Files.newInputStream(path); Workbook wb = new XSSFWorkbook(in)) {
            Sheet ws = wb.getSheet(sheet);
            if (ws == null) throw new IllegalArgumentException("Sheet not found: " + sheet);

            DataFormatter fmt = new DataFormatter();   // reads numbers and dates as shown in Excel
            Row header = ws.getRow(0);
            List<Map<String, String>> rows = new ArrayList<>();

            for (int r = 1; r <= ws.getLastRowNum(); r++) {
                Row row = ws.getRow(r);
                if (row == null) continue;
                Map<String, String> map = new LinkedHashMap<>();
                for (int c = 0; c < header.getLastCellNum(); c++)
                    map.put(fmt.formatCellValue(header.getCell(c)), fmt.formatCellValue(row.getCell(c)));
                rows.add(map);
            }
            return rows;
        }
    }
}
```

Code: data-driven tests

testdata/json/login-cases.json (wrong passwords are not secrets)

```
[
  { "name": "empty email",    "email": "",                  "password": "any-value",      "expected": "Email is required" },
  { "name": "invalid format", "email": "not-an-email",      "password": "any-value",      "expected": "Enter a valid email" },
  { "name": "wrong password", "email": "standard@shop.example", "password": "wrong-password", "expected": "Email or password is incorrect" }
]
```

C#

```
public sealed record LoginCase(string Name, string Email, string Password, string Expected);

public sealed class LoginValidationTests : WebTestBase   // gives Page and LoginPage
{
    // One test method, many rows of data
    private static IEnumerable<TestCaseData> Cases() =>
        JsonData.Load<List<LoginCase>>("testdata/json/login-cases.json")
            .Select(c => new TestCaseData(c).SetName($"Login shows error: {c.Name}"));

    [TestCaseSource(nameof(Cases))]
    public async Task Login_shows_validation_message(LoginCase c)
    {
        await LoginPage.OpenAsync();
        await LoginPage.LoginAsAsync(c.Email, c.Password);
        await Expect(Page.GetByTestId("login-message")).ToHaveTextAsync(c.Expected);
    }
}
```

TypeScript

```
import { test, expect } from '../fixtures';
import cases from '../../testdata/json/login-cases.json';

// One test body, many rows of data. Each row becomes its own test.
for (const c of cases) {
  test(`login shows error: ${c.name}`, async ({ page, loginPage }) => {
    await loginPage.open();
    await loginPage.loginAs(c.email, c.password);
    await expect(page.getByTestId('login-message')).toHaveText(c.expected);
  });
}
```

Python

```
import json
from pathlib import Path

import pytest
from playwright.sync_api import expect

CASES = json.loads(Path("testdata/json/login-cases.json").read_text(encoding="utf-8"))


# One test function, many rows of data. Each row becomes its own test.
@pytest.mark.parametrize("case", CASES, ids=lambda c: c["name"])
def test_login_shows_validation_message(page, login_page, case):
    login_page.open()
    login_page.login_as(case["email"], case["password"])
    expect(page.get_by_test_id("login-message")).to_have_text(case["expected"])
```

Java

```
public record LoginCase(String name, String email, String password, String expected) {}

class LoginValidationTest extends WebTestBase {   // gives page and loginPage

    static Stream<LoginCase> cases() {
        return JsonData.loadList("testdata/json/login-cases.json", LoginCase.class).stream();
    }

    // One test method, many rows of data. Each row becomes its own test.
    @ParameterizedTest(name = "login shows error: {0}")
    @MethodSource("cases")
    void loginShowsValidationMessage(LoginCase c) {
        loginPage.open();
        loginPage.loginAs(c.email(), c.password());
        assertThat(page.getByTestId("login-message")).hasText(c.expected());
```

```
    }
}
```

Code: builder

C#

```
public sealed class OrderBuilder
{
    private string _customerId = "CUST-DEFAULT";      // safe defaults
    private readonly List<OrderItem> _items = [];

    public OrderBuilder ForCustomer(string id) { _customerId = id; return this; }
    public OrderBuilder WithItem(string sku, int qty = 1) { _items.Add(new(sku, qty)); return this; }
    public Order Build() => new(_customerId, [.. _items]);
}

// The test only sets what it cares about
var order = new OrderBuilder().ForCustomer(customer.Id).WithItem("SKU-MOUSE-01", 2).Build();
```

TypeScript

```
export class OrderBuilder {
  private customerId = 'CUST-DEFAULT';   // safe defaults
  private items: OrderItem[] = [];

  forCustomer(id: string): this { this.customerId = id; return this; }
  withItem(sku: string, quantity = 1): this { this.items.push({ sku, quantity }); return this; }
  build(): Order { return { customerId: this.customerId, items: [...this.items] }; }
}

// The test only sets what it cares about
const order = new OrderBuilder().forCustomer(customer.id).withItem('SKU-MOUSE-01', 2).build();
```

Python

```
class OrderBuilder:
    def __init__(self) -> None:
        self.customer_id = "CUST-DEFAULT"   # safe defaults
        self.items: list[dict] = []

    def for_customer(self, customer_id: str) -> "OrderBuilder":
        self.customer_id = customer_id
        return self

    def with_item(self, sku: str, quantity: int = 1) -> "OrderBuilder":
        self.items.append({"sku": sku, "quantity": quantity})
        return self

    def build(self) -> dict:
        return {"customerId": self.customer_id, "items": list(self.items)}


# The test only sets what it cares about
order = OrderBuilder().for_customer(customer["id"]).with_item("SKU-MOUSE-01", 2).build()
```

Java

```
public final class OrderBuilder {
    private String customerId = "CUST-DEFAULT";   // safe defaults
    private final List<OrderItem> items = new ArrayList<>();

    public OrderBuilder forCustomer(String id) { this.customerId = id; return this; }
    public OrderBuilder withItem(String sku, int qty) { items.add(new OrderItem(sku, qty)); return this; }
    public Order build() { return new Order(customerId, List.copyOf(items)); }
}
```

```
// The test only sets what it cares about
Order order = new OrderBuilder().forCustomer(customer.id()).withItem("SKU-MOUSE-01", 2).build();
```

Watch out

- Every test must be able to run alone, in any order and in parallel. Create **unique data per test** (see topic 20).
- Clean up what you create, or use data that expires on its own.
- Use a test domain such as `example.test`. Never use real customer data or real mailboxes.

17 Hooks, fixtures and the test lifecycle

Hooks set things up before a test and clean up after it, even when the test fails.

What it is

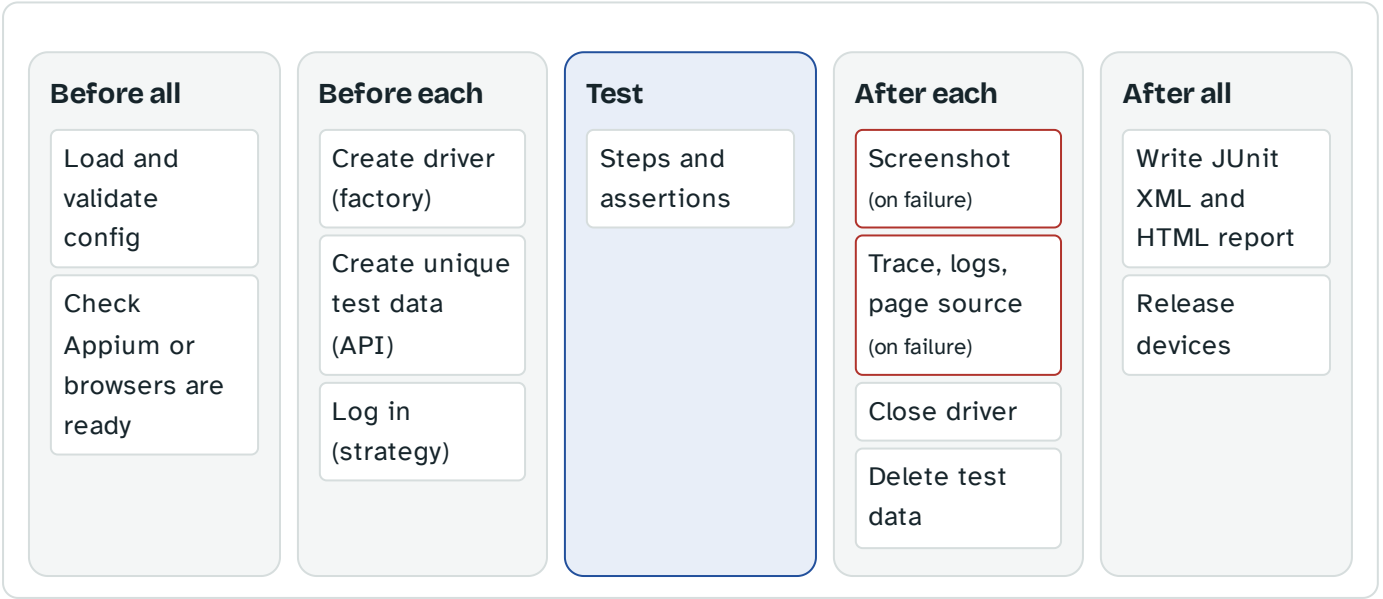
A hook is code that the runner calls at a fixed moment: before all tests, before each test, after each test, after all tests. Each runner names them differently:

Moment	C# (NUnit / Reqnroll)	TypeScript (Playwright)	Python (pytest)	Java (JUnit 5 / Cucumber)
Once before all	[OneTimeSetUp] / [BeforeTestRun]	globalSetup, worker fixture	session-scoped fixture	@BeforeAll
Before each	[SetUp] / [BeforeScenario]	test fixture, beforeEach	function-scoped fixture	@BeforeEach / @Before
After each	[TearDown] / [AfterScenario]	code after `use()`, afterEach	code after `yield`	@AfterEach / @After
Once after all	[OneTimeTearDown] / [AfterTestRun]	globalTeardown	end of session fixture	@AfterAll

Why you need it

If cleanup lives inside the test, it never runs when the test fails early. Then drivers stay open, devices stay locked and the next test fails too. Hooks always run, so evidence is captured and resources are released.

See it: one test's lifecycle



Code: driver per test, screenshot on failure, always clean up

```C#
// NUnit: one driver per test, evidence on failure, always clean up
public abstract class BaseTest
{
    protected IUiDriver Ui = null!;
```

```
    [SetUp]                                  // before each test
    public async Task StartDriver() => Ui = await DriverFactory.CreateAsync(TestConfig.Current);

    [TearDown]                               // after each test
    public async Task StopDriver()
    {
        try
        {
            if (TestContext.CurrentContext.Result.Outcome.Status == TestStatus.Failed)
            {
                var file = Path.Combine("output", "screenshots", $"
{TestContext.CurrentContext.Test.Name}.png");
                Directory.CreateDirectory(Path.GetDirectoryName(file)!);
                await File.WriteAllBytesAsync(file, await Ui.ScreenshotAsync());
                TestContext.AddTestAttachment(file);
            }
        }
        finally
        {
            await Ui.DisposeAsync();             // runs even if the screenshot fails
        }
    }
}
```

TypeScript

```
// tests/fixtures.ts: a fixture is a hook with setup and teardown in one place
import { test as base } from '@playwright/test';

export const test = base.extend<{ ui: UiDriver }>({
  ui: async ({}, use, testInfo) => {
    const ui = await createDriver(config.platform);   // before each test
    try {
      await use(ui);                                   // the test runs here
      if (testInfo.status !== testInfo.expectedStatus) {
        await testInfo.attach('screenshot', { body: await ui.screenshot(), contentType: 'image/png' });
      }
    } finally {
      await ui.quit();                                 // always clean up
    }
  },
});
export { expect } from '@playwright/test';
```

Python

```
# tests/conftest.py
from pathlib import Path

import pytest

OUTPUT = Path("output")


@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    """Remember each phase's result so fixtures can see if the test failed."""
    outcome = yield
    report = outcome.get_result()
    setattr(item, f"rep_{report.when}", report)


@pytest.fixture
def ui(request):
    driver = create_driver(get_settings())          # before each test
    yield driver                                    # the test runs here
    try:
        report = getattr(request.node, "rep_call", None)
        if report and report.failed:
```

```
            path = OUTPUT / "screenshots" / f"{request.node.name}.png"
            path.parent.mkdir(parents=True, exist_ok=True)
            path.write_bytes(driver.screenshot())
    finally:
        driver.quit()                              # always clean up
```

Java

```java
// Runs right after the test body, BEFORE @AfterEach closes the driver
public final class EvidenceOnFailure implements AfterTestExecutionCallback {
    @Override
    public void afterTestExecution(ExtensionContext ctx) throws Exception {
        if (ctx.getExecutionException().isEmpty()) return;      // test passed
        BaseTest test = (BaseTest) ctx.getRequiredTestInstance();
        Path file = Path.of("output", "screenshots", ctx.getRequiredTestMethod().getName() + ".png");
        Files.createDirectories(file.getParent());
        Files.write(file, test.ui.screenshot());
    }
}

@ExtendWith(EvidenceOnFailure.class)
public abstract class BaseTest {
    protected UiDriver ui;

    @BeforeEach                                    // before each test
    void startDriver() throws Exception { ui = DriverFactory.create(Config.get()); }

    @AfterEach                                     // after each test
    void stopDriver() throws Exception { ui.close(); }
}
```

Watch out

- Hooks set up and clean up. They do not contain business assertions.
- Capture the screenshot **before** you close the driver. Order matters.
- Use the runner's built-in evidence when it has it (Playwright's `screenshot: 'only-on-failure'`). Do not add a second screenshot hook next to it.

18 BDD: feature files and step definitions

Gherkin describes behavior in business words. Step definitions translate those words into framework calls.

What it is

BDD (Behavior-Driven Development) is a way to agree on behavior using examples. The examples are written in Gherkin: `Given` a starting state, `When` an action, `Then` an expected result. Tools such as Reqnroll (C#), Cucumber (TypeScript and Java) and pytest-bdd (Python) connect each line to a step definition.

Why (and when) you need it

Use BDD when product, QA and developers review the examples together. Then the feature file is a shared, living specification. If only engineers read the tests, BDD adds a layer you must maintain without adding value. Start with the normal runner and add BDD later if you need it.

See it: how a line becomes an action

Feature file	`When the customer logs in with valid credentials`
Step definition (thin)	`loginFacade.loginAs(user)`
Facade	`strategy.login(user)`
Page	`type email, type password, click`
Driver	`Playwright or Appium`

Code: the feature file

tests/features/customer-login.feature

```
Feature: Customer login

  Scenario: Registered customer logs in successfully
    Given the customer account "standard" exists
    When the customer logs in with valid credentials
    Then the dashboard should be displayed

  Scenario Outline: Login form shows a clear error
    When the customer logs in with email "<email>" and password "<password>"
    Then the login error "<message>" should be shown

    Examples:
      | email                | password       | message                       |
      |                      | any-value      | Email is required             |
      | standard@shop.example | wrong-password | Email or password is incorrect |
```

Code: step definitions

C#

```
// Reqnroll: bindings only translate words into framework calls
[Binding]
public sealed class LoginSteps(ScenarioSession session, UserRepository users)
{
```

```
    [Given("the customer account {string} exists")]
    public void GivenAccountExists(string key) => session.CurrentUser = users.Get(key);

    [When("the customer logs in with valid credentials")]
    public Task WhenCustomerLogsIn() => session.Login.LoginAsAsync(session.CurrentUser!);

    [Then("the dashboard should be displayed")]
    public Task ThenDashboardIsDisplayed() => session.Dashboard.ExpectLoadedAsync();
}
```

TypeScript

```
// @cucumber/cucumber: "this" is the World, created fresh for each scenario
import { Given, When, Then } from '@cucumber/cucumber';

Given('the customer account {string} exists', function (this: ShopWorld, key: string) {
  this.user = this.users.get(key);
});

When('the customer logs in with valid credentials', async function (this: ShopWorld) {
  await this.loginFacade.loginAs(this.user!);
});

Then('the dashboard should be displayed', async function (this: ShopWorld) {
  await this.dashboard.expectLoaded();
});
```

Python

```
# pytest-bdd: steps reuse normal pytest fixtures
from pytest_bdd import given, parsers, scenarios, then, when

scenarios("../features/customer-login.feature")


@given(parsers.parse('the customer account "{key}" exists'), target_fixture="user")
def account_exists(users, key):
    return users.get(key)


@when("the customer logs in with valid credentials")
def logs_in(login_facade, user):
    login_facade.login_as(user)


@then("the dashboard should be displayed")
def dashboard_shown(dashboard):
    dashboard.expect_loaded()
```

Java

```
// Cucumber-JVM with cucumber-picocontainer: a new ScenarioContext per scenario
public class LoginSteps {
    private final ScenarioContext ctx;
    public LoginSteps(ScenarioContext ctx) { this.ctx = ctx; }

    @Given("the customer account {string} exists")
    public void accountExists(String key) { ctx.user = ctx.users.get(key); }

    @When("the customer logs in with valid credentials")
    public void logsIn() { ctx.loginFacade.loginAs(ctx.user); }

    @Then("the dashboard should be displayed")
    public void dashboardShown() { ctx.dashboard.expectLoaded(); }
}
```

Watch out

- Write behavior, not clicks. "The customer logs in" is good. "Click #submit" is not. Feature files never contain selectors.
- One behavior per scenario. Keep `Background` short.
- No static or global scenario state. Scenarios may run in parallel. Use the scenario context or world object.
- One phrase per action. Do not write three different sentences for the same step.

19 API layer: clients, checks and fast setup

API clients test the backend directly, and they prepare data for UI tests in milliseconds.

What it is

An API client is like a page object for an endpoint group. It knows paths, headers and status codes. The test checks the business meaning. API tests check status codes, promised fields and types, business rules, authorization and error responses.

Why you need it

- API tests are much faster and steadier than UI tests, so business rules belong here.
- Creating a user and an order through the API takes a fraction of a second. Creating them through five screens is slow and adds five extra chances to fail.

Code: client plus API setup for a UI test

C#

```
public sealed class OrdersApi(IAPIRequestContext request)
{
    public async Task<Order> CreateAsync(Order order)
    {
        var response = await request.PostAsync("/orders", new() { DataObject = order });
        // The status code is part of the endpoint's contract, so the client checks it
        if (response.Status != 201)
            throw new InvalidOperationException($"Create order failed ({response.Status}): {await
response.TextAsync()}");
        return (await response.JsonAsync<Order>())!;
    }
}

// API setup for a UI test: fast and stable
var customer = await usersApi.CreateAsync(UniqueData.Customer());
var order = await ordersApi.CreateAsync(new OrderBuilder().ForCustomer(customer.Id).WithItem("SKU-MOUSE-
01").Build());
```

TypeScript

```
import { APIRequestContext } from '@playwright/test';

export class OrdersApi {
  constructor(private readonly request: APIRequestContext) {}

  async create(order: Order): Promise<Order> {
    const response = await this.request.post('/orders', { data: order });
    // The status code is part of the endpoint's contract, so the client checks it
    if (response.status() !== 201) {
      throw new Error(`Create order failed (${response.status()}): ${await response.text()}`);
    }
    return response.json();
  }
}

// API setup for a UI test: fast and stable
const customer = await usersApi.create(uniqueCustomer());
const order = await ordersApi.create(new OrderBuilder().forCustomer(customer.id).withItem('SKU-MOUSE-
01').build());
```

Python

```
class OrdersApi:
    def __init__(self, request) -> None:
        self.request = request  # Playwright APIRequestContext
```

```python
    def create(self, order: dict) -> dict:
        response = self.request.post("/orders", data=order)
        # The status code is part of the endpoint's contract, so the client checks it
        if response.status != 201:
            raise AssertionError(f"Create order failed ({response.status}): {response.text()}")
        return response.json()


# API setup for a UI test: fast and stable
customer = users_api.create(unique_customer())
order = orders_api.create(OrderBuilder().for_customer(customer["id"]).with_item("SKU-MOUSE-01").build())
```

Java

```java
public final class OrdersApi {
    private final APIRequestContext request;
    private final ObjectMapper json = new ObjectMapper();

    public OrdersApi(APIRequestContext request) { this.request = request; }

    public Order create(Order order) throws IOException {
        APIResponse response = request.post("/orders", RequestOptions.create().setData(order));
        // The status code is part of the endpoint's contract, so the client checks it
        if (response.status() != 201)
            throw new IllegalStateException("Create order failed (" + response.status() + "): " +
response.text());
        return json.readValue(response.body(), Order.class);
    }
}

// API setup for a UI test: fast and stable
Customer customer = usersApi.create(UniqueData.customer());
Order order = ordersApi.create(new OrderBuilder().forCustomer(customer.id()).withItem("SKU-MOUSE-01",
1).build());
```

Watch out

- Do not check fields the API documentation does not promise. Do not depend on list order unless it is guaranteed.
- Add schema checks. They catch contract changes even when the business logic still passes.
- Do not mock everything. A fully mocked suite can pass while the real services cannot talk to each other. Add contract tests between services.

20 Utilities: small, focused helpers

Utilities are small tools many layers need: unique data, polling, logging and secret masking.

What belongs here

Belongs in utils	Does not belong in utils
Unique ids and emails	Locators (they go in pages)
Polling with a deadline	Business flows (they go in facades)
Secret masking for logs	Test data (it goes in repositories)
Date and time helpers with a fixed timezone	Waits for elements (they go in the driver adapter)
A structured logger	"Misc" functions nobody can name

Why you need it

Small shared helpers keep tests short and consistent. Keep them small, though. A `utils` folder with no rules turns into a junk drawer.

Code: unique data and secret masking

C#

```
public static class Unique
{
    public static string Id(string prefix) => $"{prefix}-{Guid.NewGuid():N}"[..(prefix.Length + 9)];
    public static string Email(string tag = "user") => $"{tag}-{Guid.NewGuid():N}@example.test";
}

public static class Secrets
{
    private static readonly Regex SecretKey = new("password|token|authorization|cookie|secret",
RegexOptions.IgnoreCase);

    // Mask secrets before anything is logged or attached to a report
    public static Dictionary<string, object?> Mask(IDictionary<string, object?> data) =>
        data.ToDictionary(kv => kv.Key, kv => SecretKey.IsMatch(kv.Key) ? "***" : kv.Value);
}

// Every test gets its own data, so tests never fight over one account
var email = Unique.Email("checkout");   // checkout-3f9c...@example.test
```

TypeScript

```
import { randomUUID } from 'node:crypto';

export const unique = {
  id: (prefix: string) => `${prefix}-${randomUUID().slice(0, 8)}`,
  email: (tag = 'user') => `${tag}-${randomUUID()}@example.test`,
};

const SECRET_KEY = /password|token|authorization|cookie|secret/i;

// Mask secrets before anything is logged or attached to a report
export function mask(data: Record<string, unknown>): Record<string, unknown> {
  return Object.fromEntries(
    Object.entries(data).map(([key, value]) => [key, SECRET_KEY.test(key) ? '***' : value]),
  );
}
```

```
// Every test gets its own data, so tests never fight over one account
const email = unique.email('checkout'); // checkout-3f9c...@example.test
```

Python

```
import re
from uuid import uuid4

SECRET_KEY = re.compile(r"password|token|authorization|cookie|secret", re.IGNORECASE)


def unique_id(prefix: str) -> str:
    return f"{prefix}-{uuid4().hex[:8]}"


def unique_email(tag: str = "user") -> str:
    return f"{tag}-{uuid4().hex}@example.test"


def mask(data: dict) -> dict:
    """Mask secrets before anything is logged or attached to a report."""
    return {k: "***" if SECRET_KEY.search(k) else v for k, v in data.items()}


# Every test gets its own data, so tests never fight over one account
email = unique_email("checkout")  # checkout-3f9c...@example.test
```

Java

```
public final class Unique {
    private Unique() {}
    public static String id(String prefix) { return prefix + "-" + UUID.randomUUID().toString().substring(0,
8); }
    public static String email(String tag) { return tag + "-" + UUID.randomUUID() + "@example.test"; }
}

public final class Secrets {
    private static final Pattern SECRET_KEY =
        Pattern.compile("password|token|authorization|cookie|secret", Pattern.CASE_INSENSITIVE);

    // Mask secrets before anything is logged or attached to a report
    public static Map<String, Object> mask(Map<String, Object> data) {
        Map<String, Object> out = new LinkedHashMap<>();
        data.forEach((k, v) -> out.put(k, SECRET_KEY.matcher(k).find() ? "***" : v));
        return out;
    }
}

// Every test gets its own data, so tests never fight over one account
String email = Unique.email("checkout");   // checkout-3f9c...@example.test
```

Watch out

- Utilities have no hidden state. A helper that remembers the "current user" breaks parallel runs.
- Framework utilities deserve their own unit tests, because every test depends on them.

21 Locator strategy for web, mobile and desktop

A good locator finds the element by its purpose, not by its position on the screen.

What it is

A locator tells the tool which element to use. Some locators survive redesigns. Others break when a developer adds one `<div>`. Each platform has its own best options.

Why you need it

Weak locators are a top cause of flaky and broken tests. A positional XPath like `//div[3]/div[2]/button` breaks when the layout changes, even if the feature still works.

See it: priority by platform

Web

1. **Role and accessible name**
   `getByRole('button', { name: 'Log in' })`
   What users and screen readers see. Also catches accessibility bugs.
2. **Label**
   `getByLabel('Email address')`
   Best for form fields.
3. **Test id**
   `getByTestId('login-button')`
   Stable when text is translated or the widget is complex.
4. **Stable visible text**
   `getByText('Order placed')`
   Fine for messages; breaks when wording changes.
5. **Stable CSS attribute**
   `[name="email"]`
   Only attributes that are part of the contract.
6. **XPath (last resort)**
   `//form//button[@type="submit"]`
   Never positional like //div[3]/div[2]/button.

Android

1. **Accessibility id (content-desc)**
   `~login-button`
   Fast and cross-platform. Remember screen readers read it aloud.
2. **Resource id**
   `id=com.shop:id/login_button`
   Stable and invisible to users. A good test-only name.
3. **UiAutomator selector**
   `android=new UiSelector().text("Log in")`
   Powerful, but Android-only.
4. **XPath (last resort)**
   `//android.widget.Button[@text="Log in"]`
   Slow and fragile on large screens.

iOS

1. **Accessibility id**
   `~login-button`
   Set with accessibilityIdentifier. Not read aloud.
2. **Predicate string**
   `-ios predicate string:name == "login-button"`
   Fast native query.

Desktop

1. **AutomationId (accessibility id)**
   `~LoginButton`
   Set by developers in XAML or WinForms. The best choice.
2. **Name**
   `name=Log in`
   Changes with language and wording.

3. **Class chain**
   `-ios class chain:**/XCUIElementTypeButton[1]`
   Fast; avoid index-based chains when you can.
4. **XPath (last resort)**
   `//XCUIElementTypeButton[@name="Log in"]`
   Very slow on iOS.

3. **Class name with name**
   `ClassName=Button plus name`
   Use when there is no AutomationId.
4. **XPath (last resort)**
   `//Button[@Name="Log in"]`
   Slow and fragile.

How test IDs get added

1. QA names the element in the ticket and proposes an id that follows the rule: lowercase, hyphens, describes the purpose, such as `checkout-submit`.
2. The developer adds the attribute in the same pull request as the feature: `data-testid` on web, `accessibilityIdentifier` on iOS, `contentDescription` or a resource id on Android, `AutomationId` on Windows.
3. The page object uses the id in that same pull request.
4. Adding test ids is part of the definition of done for any new screen.

Watch out

! Avoid generated class names, positional XPath and anything tied to layout.

! On Android, `contentDescription` is also read aloud by screen readers. Do not put technical ids where users will hear them. Agree on resource ids for test-only names.

! Web semantic locators (role and label) also catch accessibility problems. Test ids are better for localized text and complex widgets. Record your team's choice in an ADR (Architecture Decision Record).

22 Anti-flakiness: why tests pass, then fail, with no code change

A flaky test sometimes passes and sometimes fails on the same code. Flaky tests destroy trust faster than missing tests.

What it is

A test is flaky when its result depends on something other than the product: timing, shared data, test order, the network or the machine. The fix is almost never "add a longer sleep." The fix is to remove the hidden dependency.

Why it matters

When a suite is flaky, people stop reading failures. They click "re-run" until it turns green. Then a real bug fails the build and everyone re-runs that too. A flaky test is not a small problem. It trains the team to ignore the suite.

See it: hard sleep vs waiting for state

Both tests click "Save," then check for "Saved." The server's response time changes a little on every run, like a real environment. Move the slider to make the server slower, then run 30 times.

Both tests click "Save," then check for "Saved." Test A sleeps a fixed 2 seconds. Test B waits up to 10 seconds for the message to appear.

Server response on this run	Test A: hard sleep 2 s	Test B: wait for state
0.8 s	**Pass, but wasted 1.2 s**	**Pass, continued after 0.8 s**
1.7 s	**Pass**	**Pass, continued after 1.7 s**
2.3 s	**Fail**	**Pass, continued after 2.3 s**
3.6 s	**Fail**	**Pass, continued after 3.6 s**

On a normal day, response times move around. Test A passes on fast runs and fails on slow ones: it is flaky, though nothing in the product changed. Test B is stable, and it is also faster when the app is fast. Open the HTML version to run this simulation live.

See it: the common causes and their fixes

Fixed sleeps

Symptom: Passes locally, fails in CI.

Fix: Wait for a visible state or a network response.

Async backend work

Symptom: Order status is still "Processing" when checked.

Fix: Poll the status with a deadline.

Animations

Symptom: Click lands on a moving element.

Fix: Wait for stability; ask for a test mode that turns animations off.

Weak locators

Symptom: Breaks after an unrelated layout change.

Fix: Role, label, test id or accessibility id.

Shared accounts and data

Symptom: Fails only when run together with other tests.

Fix: Unique data per test, created through the API.

Test order dependence

Symptom: Passes in order, fails alone or in random order.

Fix: Each test sets up everything it needs.

Unseeded randomness

Symptom: Fails one run in fifty, cannot reproduce.

Fix: Seed random values and log the seed.

Timezones and dates

Symptom: Fails near midnight or month end.

Fix: Fix the timezone and inject the clock.

Third-party services

Symptom: Payment sandbox or email is slow or down.

Fix: Mock at the boundary; keep a few real contract checks.

Devices and emulators

Symptom: Slow boot, keyboard hides a button, low memory.

Fix: Warm devices, hide the keyboard, one device per worker.

The core rules

1. **Never use fixed sleeps.** Enforce it with a lint rule or a CI check.
2. **Wait for state, not time:** a visible message, an enabled button, a network response.
3. **Use stable locators** (topic 21).
4. **Give each test its own data** and do not let tests depend on each other.
5. **Set up through the API,** not the UI.
6. **Retry assertions, never actions.** The framework can check "is it Completed yet?" many times. It must never click "Place order" twice.
7. **Fail fast.** Check that the page is ready before you start. A clear early failure beats a confusing late one.
8. **Control the environment:** fixed timezone, seeded random values, animations disabled in test mode.

23 Waiting and synchronization, in code

Each tool waits differently. Your framework should make the right wait the easy default.

What it is

Playwright already waits before it clicks or types. It checks that the element is visible, stable, enabled and able to receive events. Appium and Selenium do not. They act immediately unless you add an explicit wait. This is why a hybrid framework puts waits inside the **driver adapter**. Pages and tests then never think about timing.

Why you need it

If every test author writes their own waits, some will use sleeps, some will forget, and some will wait for the wrong thing. One adapter, written once and reviewed carefully, fixes timing for every test on that platform.

Code: web, wait for state and network

C#

```
// BAD: guesses how long the app needs
await Page.GetByRole(AriaRole.Button, new() { Name = "Save" }).ClickAsync();
await Task.Delay(3000);

// GOOD 1: wait for what the user would see
await Page.GetByRole(AriaRole.Button, new() { Name = "Save" }).ClickAsync();
await Expect(Page.GetByText("Saved successfully")).ToBeVisibleAsync();

// GOOD 2: wait for the exact response the next step needs.
// Start listening BEFORE the click, or a fast response can be missed.
var responseTask = Page.WaitForResponseAsync(r =>
    r.Request.Method == "POST" &&
    new Uri(r.Url).AbsolutePath == "/api/orders" &&
    r.Status == 201);
await Page.GetByRole(AriaRole.Button, new() { Name = "Place order" }).ClickAsync();
await responseTask;
```

TypeScript

```
// BAD: guesses how long the app needs
await page.getByRole('button', { name: 'Save' }).click();
await page.waitForTimeout(3000);

// GOOD 1: wait for what the user would see
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.getByText('Saved successfully')).toBeVisible();

// GOOD 2: wait for the exact response the next step needs.
// Start listening BEFORE the click, or a fast response can be missed.
const responsePromise = page.waitForResponse((r) =>
  r.request().method() === 'POST' &&
  new URL(r.url()).pathname === '/api/orders' &&
  r.status() === 201,
);
await page.getByRole('button', { name: 'Place order' }).click();
await responsePromise;
```

Python

```
from urllib.parse import urlparse

from playwright.sync_api import expect

# BAD: guesses how long the app needs
page.get_by_role("button", name="Save").click()
page.wait_for_timeout(3000)
```

```python
# GOOD 1: wait for what the user would see
page.get_by_role("button", name="Save").click()
expect(page.get_by_text("Saved successfully")).to_be_visible()

# GOOD 2: wait for the exact response the next step needs.
# The "with" block starts listening BEFORE the click.
with page.expect_response(
    lambda r: r.request.method == "POST"
    and urlparse(r.url).path == "/api/orders"
    and r.status == 201
):
    page.get_by_role("button", name="Place order").click()
```

Java

```java
// BAD: guesses how long the app needs
page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Save")).click();
Thread.sleep(3000);

// GOOD 1: wait for what the user would see
page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Save")).click();
assertThat(page.getByText("Saved successfully")).isVisible();

// GOOD 2: wait for the exact response the next step needs.
// Playwright starts listening, THEN runs the click you pass in.
page.waitForResponse(
    r -> r.request().method().equals("POST")
        && URI.create(r.url()).getPath().equals("/api/orders")
        && r.status() == 201,
    () -> page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Place order")).click());
```

Code: mobile and desktop, explicit waits inside the Appium adapter

C#

```csharp
using OpenQA.Selenium;
using OpenQA.Selenium.Appium;
using OpenQA.Selenium.Support.UI;

// Appium does NOT auto-wait like Playwright. This adapter adds the waits ONCE,
// so no page object or test ever needs a sleep.
public sealed class AppiumUiDriver : IUiDriver
{
    private readonly AppiumDriver _driver;
    private readonly WebDriverWait _wait;

    public AppiumUiDriver(AppiumDriver driver, int timeoutSeconds = 15)
    {
        _driver = driver;
        _wait = new WebDriverWait(driver, TimeSpan.FromSeconds(timeoutSeconds))
        {
            PollingInterval = TimeSpan.FromMilliseconds(250)
        };
        _wait.IgnoreExceptionTypes(typeof(StaleElementReferenceException));
    }

    private IWebElement Find(string id, bool mustBeEnabled) => _wait.Until(d =>
    {
        var el = d.FindElements(MobileBy.AccessibilityId(id)).FirstOrDefault(e => e.Displayed);
        return el is not null && (!mustBeEnabled || el.Enabled) ? el : null;
    });

    public Task ClickAsync(string id) { Find(id, mustBeEnabled: true).Click(); return Task.CompletedTask; }

    public Task TypeAsync(string id, string text)
    {
        var el = Find(id, mustBeEnabled: true);
```

```
        el.Clear();
        el.SendKeys(text);
        return Task.CompletedTask;
    }

    public Task<string> TextOfAsync(string id) => Task.FromResult(Find(id, false).Text);
    public Task WaitVisibleAsync(string id) { Find(id, false); return Task.CompletedTask; }
    public Task<byte[]> ScreenshotAsync() => Task.FromResult(_driver.GetScreenshot().AsByteArray);
    public ValueTask DisposeAsync() { _driver.Quit(); return ValueTask.CompletedTask; }
}
```

TypeScript

```
import type { Browser } from 'webdriverio';

// Appium does NOT auto-wait like Playwright. This adapter adds the waits ONCE,
// so no page object or test ever needs a sleep.
export class AppiumDriver implements UiDriver {
  constructor(private readonly driver: Browser, private readonly timeout = 15_000) {}

  private async ready(target: string, mustBeEnabled: boolean) {
    const el = this.driver.$(target);
    await el.waitForDisplayed({ timeout: this.timeout, timeoutMsg: `Not visible: ${target}` });
    if (mustBeEnabled) {
      await el.waitForEnabled({ timeout: this.timeout, timeoutMsg: `Not enabled: ${target}` });
    }
    return el;
  }

  async click(target: string) { await (await this.ready(target, true)).click(); }
  async type(target: string, text: string) { await (await this.ready(target, true)).setValue(text); } // clears
first
  async textOf(target: string) { return (await this.ready(target, false)).getText(); }
  async waitVisible(target: string) { await this.ready(target, false); }
  async screenshot() { return Buffer.from(await this.driver.takeScreenshot(), 'base64'); }
  async quit() { await this.driver.deleteSession(); }
}
```

Python

```
from appium.webdriver.common.appiumby import AppiumBy
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait


class AppiumDriver:
    """Appium does NOT auto-wait like Playwright. This adapter adds the waits ONCE,
    so no page object or test ever needs a sleep."""

    def __init__(self, driver, timeout: float = 15) -> None:
        self.driver = driver
        self.wait = WebDriverWait(driver, timeout, poll_frequency=0.25)

    def _visible(self, target: str):
        locator = (AppiumBy.ACCESSIBILITY_ID, target)
        return self.wait.until(EC.visibility_of_element_located(locator), f"Not visible: {target}")

    def _clickable(self, target: str):
        locator = (AppiumBy.ACCESSIBILITY_ID, target)
        return self.wait.until(EC.element_to_be_clickable(locator), f"Not clickable: {target}")

    def click(self, target: str) -> None:
        self._clickable(target).click()

    def type(self, target: str, text: str) -> None:
        element = self._clickable(target)
        element.clear()
        element.send_keys(text)
```

```
    def text_of(self, target: str) -> str:
        return self._visible(target).text

    def wait_visible(self, target: str) -> None:
        self._visible(target)

    def screenshot(self) -> bytes:
        return self.driver.get_screenshot_as_png()

    def quit(self) -> None:
        self.driver.quit()
```

Java

```
import io.appium.java_client.AppiumBy;
import io.appium.java_client.AppiumDriver;
import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

// Appium does NOT auto-wait like Playwright. This adapter adds the waits ONCE,
// so no page object or test ever needs a sleep.
public final class AppiumUiDriver implements UiDriver {
    private final AppiumDriver driver;
    private final WebDriverWait wait;

    public AppiumUiDriver(AppiumDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(15));
        wait.pollingEvery(Duration.ofMillis(250)).ignoring(StaleElementReferenceException.class);
    }

    private WebElement visible(String id) {
        return wait.withMessage("Not visible: " + id)
                   .until(ExpectedConditions.visibilityOfElementLocated(AppiumBy.accessibilityId(id)));
    }

    private WebElement clickable(String id) {
        return wait.withMessage("Not clickable: " + id)
                   .until(ExpectedConditions.elementToBeClickable(AppiumBy.accessibilityId(id)));
    }

    public void click(String id)             { clickable(id).click(); }
    public void type(String id, String text) { WebElement el = clickable(id); el.clear(); el.sendKeys(text); }
    public String textOf(String id)          { return visible(id).getText(); }
    public void waitVisible(String id)       { visible(id); }
    public byte[] screenshot()               { return driver.getScreenshotAs(OutputType.BYTES); }
    public void close()                      { driver.quit(); }
}
```

Code: polling for work that finishes later

Polling is not a blind sleep. It checks the real state, stops at a deadline and explains the failure, including the last value it saw.

C#

```
// For work that finishes later: an order moving from "Processing" to "Completed"
public static async Task PollUntilAsync<T>(
    Func<Task<T>> read, Func<T, bool> isDone, TimeSpan timeout, TimeSpan interval, string what)
{
    var deadline = DateTimeOffset.UtcNow + timeout;
    T last = default!;
    while (DateTimeOffset.UtcNow < deadline)
    {
        last = await read();
        if (isDone(last)) return;
```

```
        await Task.Delay(interval);   // controlled polling interval, not a blind sleep
    }
    throw new TimeoutException($"Timed out after {timeout} waiting for {what}. Last value: {last}");
}

await PollUntilAsync(() => ordersApi.GetStatusAsync(orderId), s => s == "Completed",
    TimeSpan.FromSeconds(60), TimeSpan.FromSeconds(1), $"order {orderId} to complete");
```

TypeScript

```
// For work that finishes later: an order moving from "Processing" to "Completed".
// expect.poll checks again and again until it passes or times out.
await expect
  .poll(() => ordersApi.getStatus(orderId), {
    message: `order ${orderId} should complete`,
    timeout: 60_000,
    intervals: [1_000],
  })
  .toBe('Completed');
```

Python

```
import time


# For work that finishes later: an order moving from "Processing" to "Completed"
def poll_until(read, is_done, timeout: float, interval: float, what: str):
    deadline = time.monotonic() + timeout
    last = None
    while time.monotonic() < deadline:
        last = read()
        if is_done(last):
            return last
        time.sleep(interval)  # controlled polling interval, not a blind sleep
    raise TimeoutError(f"Timed out after {timeout}s waiting for {what}. Last value: {last!r}")


poll_until(lambda: orders_api.get_status(order_id), lambda s: s == "Completed",
           timeout=60, interval=1, what=f"order {order_id} to complete")
```

Java

```
// For work that finishes later: an order moving from "Processing" to "Completed".
// (The Awaitility library does the same thing with a fluent API.)
public static <T> T pollUntil(Supplier<T> read, Predicate<T> isDone,
                              Duration timeout, Duration interval, String what) {
    Instant deadline = Instant.now().plus(timeout);
    T last = null;
    while (Instant.now().isBefore(deadline)) {
        last = read.get();
        if (isDone.test(last)) return last;
        try {
            Thread.sleep(interval.toMillis());   // controlled polling interval, not a blind sleep
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new IllegalStateException("Interrupted while waiting for " + what, e);
        }
    }
    throw new AssertionError("Timed out after " + timeout + " waiting for " + what + ". Last value: " + last);
}

pollUntil(() -> ordersApi.getStatus(orderId), s -> s.equals("Completed"),
          Duration.ofSeconds(60), Duration.ofSeconds(1), "order " + orderId + " to complete");
```

Watch out

- Do not wrap Playwright's `click()` in a `safeClick()` that only checks visible and enabled. Playwright already does that. Write a wrapper only if it adds team value: structured logs, domain-specific error messages or a policy.
- Match network waits on method, exact path and status. A check like `url.includes('/login')` can match unrelated requests.
- Collections: wait for the count or the first item before you use `nth(0)`.

24 Retries, flaky tracking and quarantine

Retries handle infrastructure hiccups. They must never hide product bugs.

What it is

A **retry** runs a failed test or step again. **Quarantine** moves a known-flaky test out of the blocking build, so it stops blocking pull requests. It still runs and reports in a separate job.

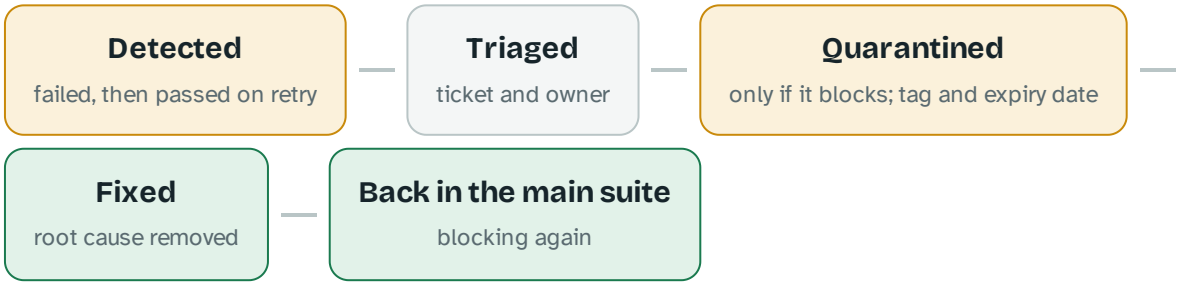
Why you need rules

Without rules, retries turn every flaky failure into a silent pass, and the flakiness grows. With rules, a test that passes only on retry is reported as **flaky**, and somebody owns fixing it.

See it: retry policy

Failure type	Retry?	Notes
Browser or device session did not start	**Yes, 1 to 2 times**	Recreate the session
Network error to the test environment	**Yes, with backoff**	Log every attempt
API returns 503 during setup	**Yes, with backoff**	Limit the attempts
Assertion failed	**No, not the whole test**	The assertion already retries until its timeout
A state-changing action (pay, submit)	**Never automatically**	Repeating it can create duplicate data
Failed, then passed on retry	**Report as flaky**	Track it until fixed

See it: a flaky test's life



Other exit: Deleted when the test adds no unique value. Do not keep it in quarantine forever.

Code: narrow retries and a quarantine tag

C#

```
// Retry INFRASTRUCTURE (starting a session), never the test's own steps.
// Note: NUnit's [Retry] re-runs a test after a failed ASSERTION, which can hide
// real bugs. Prefer this narrow retry around driver creation.
public static async Task<IUiDriver> CreateWithRetryAsync(TestConfig c, int attempts = 2)
{
    for (var attempt = 1; ; attempt++)
    {
        try { return await DriverFactory.CreateAsync(c); }
        catch (Exception ex) when (attempt < attempts && ex is WebDriverException or PlaywrightException)
        {
            Log.Warn($"Session start failed (attempt {attempt}/{attempts}): {ex.Message}");
            await Task.Delay(TimeSpan.FromSeconds(2 * attempt));   // back off
```

```
        }
    }
}

// Quarantine: a category that the blocking CI job leaves out
[Test, Category("quarantine")] // JIRA-1234, owner: checkout team, expires: 2026-11-30
public async Task Order_history_loads_after_refresh() { /* ... */ }
// Blocking job:  dotnet test --filter "TestCategory≠quarantine"
```

TypeScript

```
// playwright.config.ts
export default defineConfig({
  // Local runs never retry, so flakiness stays visible to the author.
  retries: process.env.CI ? 1 : 0,
  // The HTML report marks "failed, then passed" tests as FLAKY. Review that list.
});

// Quarantine: tag the test, run the tag in a separate, non-blocking job
test('order history loads after refresh @quarantine', async ({ page }) => {
  // QUARANTINE: JIRA-1234, owner: checkout team, expires: 2026-11-30
});

// Blocking job:    npx playwright test --grep-invert @quarantine
// Quarantine job:  npx playwright test --grep @quarantine
```

Python

```
# pip install pytest-rerunfailures
# Re-run ONLY for infrastructure errors, never for assertion failures:
#   pytest --reruns 1 --only-rerun "ConnectionError" --only-rerun "WebDriverException"

import pytest


# Quarantine: a marker that the blocking CI job leaves out
@pytest.mark.quarantine  # JIRA-1234, owner: checkout team, expires: 2026-11-30
def test_order_history_loads_after_refresh(page):
    ...

# Blocking job:    pytest -m "not quarantine"
# Quarantine job:  pytest -m quarantine
```

Java

```
// Retry INFRASTRUCTURE (starting a session), never the test's own steps.
public static UiDriver createWithRetry(Config c, int attempts) throws Exception {
    for (int attempt = 1; ; attempt++) {
        try {
            return DriverFactory.create(c);
        } catch (WebDriverException | PlaywrightException e) {
            if (attempt ≥ attempts) throw e;
            LOG.warn("Session start failed (attempt {}/{}): {}", attempt, attempts, e.getMessage());
            Thread.sleep(2000L * attempt);   // back off
        }
    }
}

// Quarantine: a tag that the blocking CI job leaves out
@Test
@Tag("quarantine") // JIRA-1234, owner: checkout team, expires: 2026-11-30
void orderHistoryLoadsAfterRefresh() { /* ... */ }

// Blocking job:    mvn test -DexcludedGroups=quarantine
// Quarantine job:  mvn test -Dgroups=quarantine
```

Watch out

- Every quarantined test has a ticket, an owner, a reason and an **expiry date**.
- Review the quarantine count and age in every planning meeting.
- A test with no unique value is deleted, not quarantined.
- Read the report, not just the exit code. A green build with ten flaky tests is not healthy.

25 Reporting, observability and the Observer pattern

A good failure answers four questions fast: what failed, where, when, and why.

What it is

Observability means a failed test leaves enough evidence to understand it without running it again: a screenshot, a trace or video, logs, a network record, plus the build, browser or device, and environment. The **Observer** pattern delivers this. When a test finishes, one event goes out. Several listeners react: one saves a screenshot, one attaches logs, one notifies the team.

Why you need it

If a failure needs a re-run to understand, people re-run instead of investigating. Good evidence makes the time to diagnose a failure drop from an hour to a minute.

See it: one event, many listeners

When a test finishes, one event goes out. Each listener decides whether to act.

Listener	Test passed	Test failed	Smoke test failed
ScreenshotOnFailure	Ignores	**Acts**	**Acts**
TraceAndLogCollector	Ignores	**Acts**	**Acts**
ReportWriter (JUnit, HTML)	**Acts**	**Acts**	**Acts**
MetricsRecorder (duration, retries)	**Acts**	**Acts**	**Acts**
ChatNotifier	Ignores	Ignores	**Acts**

How to do it

1. Send everything to one folder: `output/junit`, `output/html`, `output/test-results`, `output/logs`.
2. Use **JUnit XML** as the one machine-readable format for CI, plus an HTML or Allure report for people.
3. Capture heavy evidence (video, trace) **on failure only**, to save time and storage.
4. Track metrics over time: first-attempt pass rate, flaky rate, suite duration, quarantine count and age.

Code: observer for test events

Your runner already uses this pattern. Playwright reporters, pytest hooks, NUnit event listeners and JUnit's `TestExecutionListener` are all observers. Plug into those first. The code below shows the idea in its simplest form.

C#

```
public sealed record TestFinished(string Name, bool Passed, Exception? Error);

// Each listener reacts to the same event in its own way
public interface ITestListener { Task OnFinishedAsync(TestFinished e, IUiDriver ui); }

public sealed class TestEvents
{
    private readonly List<ITestListener> _listeners = [];
```

```
    public void Subscribe(ITestListener l) => _listeners.Add(l);

    public async Task PublishAsync(TestFinished e, IUiDriver ui)
    {
        foreach (var l in _listeners)
        {
            try { await l.OnFinishedAsync(e, ui); }
            catch (Exception ex) { Log.Warn($"Listener {l.GetType().Name} failed: {ex.Message}"); }
        }
    }
}

public sealed class ScreenshotOnFailure : ITestListener { /* save screenshot if !e.Passed */ }
public sealed class ConsoleLogCollector : ITestListener { /* attach logs if !e.Passed */ }
public sealed class ChatNotifier : ITestListener { /* message the team for failed smoke tests */ }
```

TypeScript

```
type TestFinished = { name: string; passed: boolean; error?: Error };

// Each listener reacts to the same event in its own way
interface TestListener { onFinished(e: TestFinished, ui: UiDriver): Promise<void>; }

export class TestEvents {
  private listeners: TestListener[] = [];
  subscribe(l: TestListener) { this.listeners.push(l); }

  async publish(e: TestFinished, ui: UiDriver) {
    for (const l of this.listeners) {
      try { await l.onFinished(e, ui); }
      catch (err) { console.warn(`Listener ${l.constructor.name} failed`, err); } // never break the run
    }
  }
}

class ScreenshotOnFailure implements TestListener { async onFinished() { /* ... */ } }
class ConsoleLogCollector implements TestListener { async onFinished() { /* ... */ } }
class ChatNotifier implements TestListener { async onFinished() { /* failed smoke only */ } }
```

Python

```
from dataclasses import dataclass
from typing import Protocol


@dataclass
class TestFinished:
    name: str
    passed: bool
    error: Exception | None = None


# Each listener reacts to the same event in its own way
class TestListener(Protocol):
    def on_finished(self, event: TestFinished, ui: UiDriver) -> None: ...


class TestEvents:
    def __init__(self) -> None:
        self.listeners: list[TestListener] = []

    def subscribe(self, listener: TestListener) -> None:
        self.listeners.append(listener)

    def publish(self, event: TestFinished, ui: UiDriver) -> None:
        for listener in self.listeners:
            try:
                listener.on_finished(event, ui)
```

```
            except Exception as exc:  # one broken listener must not break the run
                log.warning("Listener %s failed: %s", type(listener).__name__, exc)
```

Java

```
public record TestFinished(String name, boolean passed, Throwable error) {}

// Each listener reacts to the same event in its own way
public interface TestListener { void onFinished(TestFinished e, UiDriver ui); }

public final class TestEvents {
    private final List<TestListener> listeners = new ArrayList<>();

    public void subscribe(TestListener l) { listeners.add(l); }

    public void publish(TestFinished e, UiDriver ui) {
        for (TestListener l : listeners) {
            try { l.onFinished(e, ui); }
            catch (RuntimeException ex) {   // one broken listener must not break the run
                LOG.warn("Listener {} failed: {}", l.getClass().getSimpleName(), ex.getMessage());
            }
        }
    }
}

final class ScreenshotOnFailure implements TestListener { /* ... */ }
final class ChatNotifier implements TestListener { /* failed smoke only */ }
```

Watch out

> **!** Traces, screenshots, videos and HAR files can contain tokens and personal data. Mask cookies, authorization headers and credentials.

> **!** A broken listener must never fail the test run. Catch and log its errors.

26 Parallel execution and CI/CD

Parallel runs make the suite fast. CI makes it run on every change. Both need isolated tests first.

What it is

Parallel execution runs tests at the same time on several workers. **Sharding** splits the suite across several machines. **CI/CD** runs the right tests at the right moment: on each pull request, on merge, on a schedule and after deployment.

Why you need it

Feedback that takes hours is ignored. Feedback in minutes changes how developers work. Parallel runs are also an architecture test: they expose every bit of shared state you forgot about.

See it: estimate your run time

A simple model: total test time divided by workers, plus fixed overhead. Real runs are a little slower, because tests are not all the same length.

Run time = (tests × seconds per test) ÷ workers + overhead

Example: 600 tests, 20 seconds each, 120 seconds of overhead.

Workers	Estimated run time
1	3 h 22 min
2	1 h 42 min
4	52 min
8	27 min
16	14 min 30 s

See it: a typical pipeline

Pull request

Install (pinned) — Build, lint, type-check — API and component tests — UI smoke (blocking) — Publish evidence

Merge to main

Smoke — Broader regression — Publish report

Nightly

Full browser and device matrix — Random test order — Quarantine job — Flaky report

After deploy

Read-only synthetic checks — Approved accounts only

Manual

Chosen environment, browser, device or tag

☐ A failure blocks the merge

Parallel safety checklist

- Unique data per test, and no test depends on another.
- A browser context, device or app session per test or per worker.
- Output file names that cannot collide between workers.
- Locks or pools for shared resources: one payment sandbox, a few promo codes.
- Worker count matched to what the environment, the devices and the backend can handle.

Code: turn on parallel runs

C#

```
// AssemblyInfo.cs: run test CLASSES in parallel, 4 at a time.
// Tests inside one class run one after another, so instance fields are safe.
[assembly: Parallelizable(ParallelScope.Fixtures)]
[assembly: LevelOfParallelism(4)]

// Want tests inside one class to run in parallel too? Then each test needs
// its own instance, or they will share the same Ui field:
// [Parallelizable(ParallelScope.All)]
// [FixtureLifeCycle(LifeCycle.InstancePerTestCase)]
```

TypeScript

```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true,                        // every test can run alongside others
  workers: process.env.CI ? 4 : undefined,     // undefined = half your CPU cores locally
});

// Split one suite across 4 CI machines:
//   npx playwright test --shard=1/4   (machine 1)
//   npx playwright test --shard=2/4   (machine 2) ...
```

Python

```
# pip install pytest-xdist
#   pytest -n auto                 one worker per CPU core
#   pytest -n 4 --dist loadfile    4 workers; tests from one file stay together
#
# Each xdist worker is a separate process, so module-level state is NOT shared.
# Shared resources (one payment sandbox) still need a lock or a pool.
```

Java

```
# src/test/resources/junit-platform.properties
# Run test CLASSES in parallel; methods inside one class stay in order.
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = same_thread
junit.jupiter.execution.parallel.mode.classes.default = concurrent
junit.jupiter.execution.parallel.config.strategy = fixed
junit.jupiter.execution.parallel.config.fixed.parallelism = 4
```

Code: a CI workflow

.github/workflows/tests.yml (TypeScript shown; swap the install and run steps for your language)

```yaml
name: tests
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  smoke:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    env:
      BASE_URL: ${{ vars.BASE_URL }}
      API_BASE_URL: ${{ vars.API_BASE_URL }}
      TEST_USER_STANDARD_PASSWORD: ${{ secrets.TEST_USER_STANDARD_PASSWORD }}
    steps:
      - uses: actions/checkout@v4          # pin to the current major version
      - uses: actions/setup-node@v4
        with:
          node-version-file: .nvmrc
          cache: npm
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test --grep @smoke --grep-invert @quarantine
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}            # upload evidence even when tests fail
        with:
          name: smoke-output
          path: output/
          retention-days: 14
```

Watch out

- Start with one browser and one worker. Add workers, browsers and shards only after the suite is stable.
- Pin versions: the runtime, Playwright, Appium and its drivers, and a lock file. Upgrade on purpose, in a separate branch.
- Never give untrusted pull requests (from forks) access to secrets or deployment credentials.

27 Security and production safety

Test code touches real systems and real credentials. Treat it like production code.

Rules

- Use synthetic test data only. Never use copies of real customer data.
- Store secrets in the CI secret store or a vault, such as Azure Key Vault or AWS Secrets Manager. Prefer short-lived identities over long-lived keys.
- Treat logs, traces, screenshots, videos and HAR files as sensitive.
- Run destructive tests only in approved non-production environments.
- Run production checks only when approved: read-only, with synthetic accounts.
- Never turn off TLS certificate checks as a shortcut.
- Do not pass tokens as command-line arguments, because they appear in process lists and logs.
- Scan dependencies regularly and rotate test credentials on a schedule.

28 Review checklist and definition of done

Use this list in every pull request review. Tick items to check your own framework. Your ticks are saved in this browser.

Pull request review

- [ ] The test checks a meaningful outcome.
- [ ] It uses the lowest test level that gives enough confidence.
- [ ] Locators live in page objects, not in tests or facades.
- [ ] There is no fixed sleep anywhere.
- [ ] The test data is unique to this test.
- [ ] No state-changing action is retried automatically.
- [ ] No secrets or personal data appear in code, logs or evidence.
- [ ] A failure produces enough evidence to diagnose it.
- [ ] The test passes alone, in random order and in parallel.
- [ ] Any new abstraction solves a problem that really repeats.
- [ ] New test ids are added to the app in the same change.

Done: local use

- [ ] A new teammate can install from the README.
- [ ] One command runs the smoke tests.
- [ ] Missing or invalid config fails with a clear message.

Done: diagnostics

- [ ] Failures save evidence to output/.
- [ ] Reports show environment, platform, browser or device, duration, retries and build.
- [ ] Sensitive data is masked in logs and evidence.

Done: CI/CD

- [ ] Pull requests run a fast, blocking smoke suite.
- [ ] Broader regression runs on an agreed schedule.
- [ ] Results are published even when tests fail.
- [ ] Quarantine runs separately and is visible.

Done: maintenance

- [ ] README and architecture notes are current.
- [ ] Important decisions have ADRs.
- [ ] Runtime, tool and driver versions are pinned, with an update plan.

- [ ] Every failure and quarantined test has an owner.

Enforce what you can automatically

People forget, tools do not. Add lint rules or CI checks for: fixed sleeps, positional XPath, secret patterns in source and data files, raw driver calls inside test files, missing suite tags, and formatting and type checks.

Tool versions change. Before you start a project, check package names, driver options and CI action versions against each tool's current documentation, then pin the exact versions you use.

Original author

Hasin Md. Daiyan

Questions, corrections or ideas? Get in touch: daiyanhasin07@gmail.com